SnowPro® Core (COF-C03) — Revision Guide

Built from the official SnowPro Core COF-C03 Exam Study Guide (last updated July 8, 2026) and docs.snowflake.com. Prepared October 2026.

By **sabarishh14** · github.com/sabarishh14

> **How to use this guide.** Every section follows the official objective numbering (1.1, 1.2 …) so you can tick off the syllabus. ⭐ **= high-yield / frequently tested.** ⚠️ **= common trap.** Each domain ends with a "Must remember" box. The last part has cheat-sheet tables, the official sample questions explained, and 40 practice questions.

Contents

- **Section 0:** Exam at a glance
- **Domain 1:** Snowflake AI Data Cloud Features and Architecture (31%)
- **Domain 2:** Account Management and Data Governance (20%)
- **Domain 3:** Data Loading, Unloading, and Connectivity (18%)
- **Domain 4:** Performance Optimization, Querying, and Transformation (21%)
- **Domain 5:** Data Collaboration (10%)
- **Section 6:** Cheat sheets: numbers, "which feature?", confusing pairs, who can do what
- **Section 7:** Official sample questions, explained
- **Section 8:** Practice questions (40)
- **Section 9:** Last-48-hours checklist

---

0. Exam at a glance

Item	Detail
Exam code	**COF-C03** (replaced COF-C02; English COF-C02 retired May 14, 2026)
Questions	100, multiple choice + multiple select ("choose two/three")
Time	115 minutes (about 69 seconds per question)
Passing score	750 on a scaled 0–1000
Cost	US $175 per attempt
Delivery	Pearson VUE (online proctored or test centre)
Validity	2 years; recertify through Snowflake Continuing Education (eligible instructor-led courses, or an equal/higher SnowPro cert)
Recommended	6+ months hands-on Snowflake, basic ANSI SQL

> Question count, time and passing score come from consistent third-party sources. The official page confirms the $175 price. Re-check on the registration page.

Domain weightings (official)

Domain	Weight	≈ Questions
1.0 Snowflake AI Data Cloud Features and Architecture	**31%**	~31
2.0 Account Management and Data Governance	20%	~20
3.0 Data Loading, Unloading, and Connectivity	18%	~18
4.0 Performance Optimization, Querying, and Transformation	21%	~21
5.0 Data Collaboration	10%	~10

Not tested until globally GA (per the official guide): the default warehouse for Notebooks, and Openflow.

What changed vs COF-C02 (focus your revision here)

- 6 domains became 5. Architecture grew from about 25% to **31%**.
- New or heavier topics: **Gen2 warehouses**, **Snowflake CLI and VS Code**, **Iceberg tables**, **dynamic tables**, **Snowpipe Streaming**, **Cortex (AI SQL functions, Search, Analyst)**, **Snowflake ML**, **Notebooks and Streamlit**, **Trust Center**, **privacy policies**, **alerts and notifications**, **data lineage**, **logging and tracing**, **Git integration**, **query insights**, **query attribution**, **data clean rooms**, **Native Apps**.

Exam technique

- Read the **last line first** ("Which change reduces cost?" vs "...improves performance?"). Many options are *true statements* that don't answer *this* question.
- "Choose two/three" questions have no partial credit. Judge every option on its own.
- Phrases like **"minimize cost"**, **"without changing error handling"** and **"single option"** eliminate most choices (see official sample Q5).
- Flag and move on. Aim to finish the first pass with 20+ minutes left.

Domain 1 — Snowflake AI Data Cloud Features and Architecture (31%)

1.1 Describe and use the Snowflake architecture

Snowflake is a fully managed SaaS service on **AWS, Microsoft Azure and Google Cloud**. You don't install, configure or tune hardware. Each **account** lives in **one region on one cloud**.

⭐ **Hybrid architecture:** *shared-disk* (one central data store that every compute cluster can see) plus *shared-nothing* (each warehouse is an MPP cluster that processes its portion of data locally). The result is that **storage and compute scale independently**.

```
+------------------------ CLOUD SERVICES LAYER -------------------------+
| Authentication | Access control | Metadata management | Query parsing,  |
| optimization & compilation | Infrastructure mgmt | Transactions |       |
| Result cache | Security & key management                              |
+-----------------------------------------------------------------------+
+---------------------------- COMPUTE LAYER ----------------------------+
| Virtual warehouse A   | Virtual warehouse B   | Serverless compute       |
| (MPP cluster, local   | (isolated: no         | (Snowpipe, clustering,   |
|  SSD cache)           |  contention with A)   |  MV refresh, QAS ...)    |
+-----------------------------------------------------------------------+
+------------------------ DATABASE STORAGE LAYER -----------------------+
| Micro-partitions | columnar | compressed | encrypted (AES-256) |        |
| in cloud object storage (S3 / Blob / GCS) | reachable only through SQL |
+-----------------------------------------------------------------------+
```

Cloud Services layer ("the brain")

- Runs on compute Snowflake manages. Always on.
- Handles **authentication, access control, metadata, query parsing and optimization, infrastructure management, transaction management**, the **query result cache** and the **metadata cache**.
- ⭐ **Billing:** cloud services credits are charged **only for the part that exceeds 10% of that day's warehouse compute** (the daily adjustment).
- Queries answered entirely here need **no running warehouse**: `SHOW`, `DESCRIBE`, `SELECT COUNT(*) FROM t`, `MIN` / `MAX` on columns (from metadata), result-cache hits, `CURRENT_*()` functions.

Compute layer (virtual warehouses)

- A **virtual warehouse** is a cluster of compute resources (CPU, memory, local SSD). It runs queries, DML and data loading.
- Warehouses are **independent**: one never competes with another for resources.
- **Serverless features** use Snowflake-managed compute instead of your warehouse: Snowpipe, Snowpipe Streaming, Automatic Clustering, materialized-view maintenance, search optimization, Query Acceleration Service, serverless tasks and alerts, replication, Trust Center scans, Cortex AI.

Database Storage layer

- Loaded data is reorganised into Snowflake's **compressed, columnar, encrypted** format in **micro-partitions** in the cloud provider's object storage.
- Customers **can't see or access these files directly**, only through SQL.

- Storage is billed on the **average compressed TB per month** (on-demand or pre-purchased capacity). It includes active data, **Time Travel** data, **Fail-safe** data and files in internal stages.

⭐ Snowflake editions

Feature	Standard	Enterprise	Business Critical	VPS
Full SQL, warehouses, Snowpipe, streams, tasks, dynamic tables, Iceberg, hybrid tables, Snowpark, Streamlit, Cortex AI functions / Search / Analyst, ML functions	✅	✅	✅	✅
Time Travel	**up to 1 day**	**up to 90 days**	90	90
Fail-safe (7 days)	✅	✅	✅	✅
Network policies, MFA, SSO, OAuth, encryption of all data, resource monitors, object tags	✅	✅	✅	✅
Database and share replication, replication groups	✅	✅	✅	✅
Multi-cluster warehouses	❌	✅	✅	✅
Materialized views, Search Optimization, Query Acceleration	❌	✅	✅	✅
Column-level security (masking) and row access policies	❌	✅	✅	✅
Aggregation/projection policies, differential privacy, data classification	❌	✅	✅	✅
ACCESS_HISTORY (lineage), periodic **rekeying**	❌	✅	✅	✅
Tri-Secret Secure (customer-managed key)	❌	❌	✅	✅
Private connectivity (AWS PrivateLink, Azure Private Link, GCP Private Service Connect)	❌	❌	✅	✅
Failover/failback, client redirect, replication of account objects (users, roles, warehouses…)	❌	❌	✅	✅
HIPAA, PCI DSS, FedRAMP/ITAR support	❌	❌	✅	✅
Completely separate environment (dedicated metadata store and compute)	❌	❌	❌	✅

Memory hook: *Standard = the basics. Enterprise = "more" (more Time Travel days, more clusters, performance features, governance policies). Business Critical = "regulated and resilient" (compliance, private links, your own key, failover). VPS = "isolated".*

1.2 Use Snowflake interfaces and tools

Snowsight (web UI)

Snowsight is the default UI; the Classic Console is retired. Know where things live:

- **Projects:** Worksheets (SQL and Python), **Workspaces** (file-based IDE with Git), **Notebooks**, **Streamlit** apps, Dashboards.
- **Data / Catalog:** database object explorer, Universal Search, lineage graph, **Marketplace**, listings and Provider Studio.
- **Monitoring:** Query History, **Query Profile**, Copy History, Task History, dynamic table graph, Traces & Logs.
- **AI & ML:** Cortex playground, Cortex Search / Analyst / Agents, Snowflake Intelligence, models.
- **Admin / Governance & Security:** Cost Management (budgets, resource monitors, consumption), Warehouses, Users & Roles, **Trust Center**, Accounts (organization admin).

- Snowsight can **upload files** to stages and tables with a load wizard (up to 250 MB per file). ⚠️ `PUT` and `GET` run from a **client** (Snowflake CLI, SnowSQL, drivers), not from a worksheet.

Snowflake CLI (`snow`)

- Open-source and **the preferred command-line client**. It runs SQL *and* manages apps and objects.
- **SnowSQL** is the **legacy** CLI. It's supported until April 16, 2028 but gets no new features.
- Connections live in `~/.snowflake/config.toml` under `[connections.<name>]`.
- Command groups to recognise:
  - `snow connection add | test | list`
  - `snow sql -q "SELECT 1"` or `snow sql -f script.sql`
  - `snow object list | describe | create | drop`
  - `snow stage copy | list-files` (upload and download files)
  - `snow snowpark build | deploy` (UDFs and procedures)
  - `snow streamlit deploy`, `snow app run` (Native Apps), `snow notebook`, `snow git`, `snow spcs` (containers), `snow init` (project templates)

IDE integrations

- **Snowflake extension for Visual Studio Code:** connect, browse objects, run `.sql` files and statements, see query history and results, develop Snowpark Python.
- Other options: JetBrains / DataGrip / DBeaver through **JDBC**, Jupyter with the Python connector or Snowpark, **dbt** projects.

Other clients (also Domain 3)

JDBC, ODBC, Python, Node.js, Go, .NET and PHP PDO drivers. The **SQL API** (REST). **SnowCD** is the connectivity-diagnostic tool.

1.3 Differentiate Snowflake object hierarchy and types

```
ORGANIZATION   (ORGADMIN / GLOBALORGADMIN, organization account, ORGANIZATION_USAGE)
 +-- ACCOUNT   (account-level objects)
      +-- Users, Roles, Warehouses, Resource monitors, Network policies
      +-- Integrations (storage, API, notification, security, catalog, external access)
      +-- Shares, Applications (installed Native Apps), Compute pools, External volumes
      +-- Connections, Replication / failover groups
      +-- DATABASE
           +-- Database roles
           +-- SCHEMA
                +-- Tables, Views, Stages, File formats, Sequences, Pipes, Streams, Tasks
                    UDFs, Stored procedures, Dynamic tables, Alerts, Tags, Policies,
                    Secrets, Network rules, Models (ML), Notebooks, Streamlit apps,
                    Git repositories, Cortex Search services, Semantic views, Event tables
```

- Fully qualified name: `database.schema.object`. Unquoted identifiers are **case-insensitive and stored in UPPERCASE**. `"Quoted"` identifiers are **case-sensitive**.
- To reach a schema object you need **`USAGE` on the database and the schema** plus a privilege on the object itself.

Key database objects

Object	Remember
Stage	Location of data files, internal or external (see 3.1)
Schema	Logical container. A **managed access schema** centralises grant decisions with the schema owner.
Table	Permanent, transient, temporary, external, Iceberg, dynamic, hybrid, event (see 1.5)
View	Standard, secure, materialized, secure materialized (see 1.5)
UDF	Called inside SQL. Returns a scalar value or a table (UDTF). SQL, JavaScript, Python, Java, Scala. Can be **secure**.
File format	Named, reusable parsing rules (CSV, JSON, Avro, ORC, Parquet, XML)
Stored procedure	Run with `CALL`. Can run DDL/DML and control flow. SQL (Snowflake Scripting), JavaScript, Python, Java, Scala.
Pipe	Wraps a `COPY INTO` statement for Snowpipe
Share	Account-level object that holds grants for secure data sharing
Sequence	Unique numbers with `seq.NEXTVAL`. ⚠️ Not guaranteed gap-free. `AUTOINCREMENT` / `IDENTITY` columns use sequences.
ML model	Schema-level object in the **Model Registry**, with versions. Inference from SQL or Python.
Application	An installed **Native App** in the consumer account, created from an application package

⭐ UDF vs stored procedure

	UDF	Stored procedure
How it's called	In an expression: `SELECT f(x)`	`CALL p(x)`, as its own statement
Must return a value?	Yes (scalar or table)	Optional (single value or table)
DDL / DML inside?	No (computes values)	**Yes**
Typical use	Per-row calculations and transformations	Admin tasks, multi-step ETL, dynamic SQL
Security context	Owner (definition hidden if secure)	⭐ **Owner's rights (default)**, **caller's rights** (`EXECUTE AS CALLER`) or restricted caller's rights

⭐ **Owner's vs caller's rights** (official sample Q1):

- **Owner's rights** (default): the procedure runs with the *owner's* privileges. Callers only need `USAGE` on the procedure, not on the underlying tables. It can't read the caller's session variables or change session state.
- **Caller's rights** (`EXECUTE AS CALLER`): it runs with the *caller's* privileges, so it fails if the caller lacks `SELECT` on the table. It can use the caller's session variables and parameters.

External functions call a remote service (AWS API Gateway, Azure API Management, Google API Gateway) through an **API integration**. Newer designs use **external access integrations** from Python or Java UDFs and procedures instead.

⭐ Session and context variables, parameter hierarchy and precedence

Context functions: `CURRENT_ACCOUNT()` (locator), `CURRENT_ACCOUNT_NAME()`, `CURRENT_ORGANIZATION_NAME()`, `CURRENT_REGION()`, `CURRENT_USER()`, `CURRENT_ROLE()`, `CURRENT_SECONDARY_ROLES()`, `IS_ROLE_IN_SESSION()`, `CURRENT_WAREHOUSE()`, `CURRENT_DATABASE()`, `CURRENT_SCHEMA()`, `CURRENT_SESSION()`, `LAST_QUERY_ID()`, `INVOKER_ROLE()`.

Session variables: `SET min_amt = 100;` then `SELECT … WHERE amount > $min_amt;`. Use `IDENTIFIER($tbl)` to use a variable as an object name. Remove one with `UNSET`.

Parameter types:

Type	Set at	Examples
Account	Account only (`ALTER ACCOUNT`, ACCOUNTADMIN)	`MIN_DATA_RETENTION_TIME_IN_DAYS`, `PERIODIC_DATA_REKEYING`, `NETWORK_POLICY`
Session	Account → **User** → **Session**	`TIMEZONE`, `QUERY_TAG`, `STATEMENT_TIMEOUT_IN_SECONDS`, `USE_CACHED_RESULT`, `AUTOCOMMIT`, `DATE_INPUT_FORMAT`
Object	Account → **Database** → **Schema** → **Table** (or Account → Warehouse / User)	`DATA_RETENTION_TIME_IN_DAYS`, `MAX_DATA_EXTENSION_TIME_IN_DAYS`, `STATEMENT_TIMEOUT_IN_SECONDS` on a warehouse

⭐ **Precedence rule: the most specific level wins.** Session overrides user, which overrides account. Table overrides schema, which overrides database, which overrides account.

- See values with `SHOW PARAMETERS [LIKE '%TIMEOUT%'] IN SESSION | ACCOUNT | USER u | WAREHOUSE w | DATABASE d | TABLE t;`. The `level` column shows where each value was set.
- ⚠️ `STATEMENT_TIMEOUT_IN_SECONDS` exists at session level *and* warehouse level. **The lower non-zero value applies.** The default is 172,800 seconds (2 days).

1.4 Configure virtual warehouses

⭐ Sizes and credits (standard Gen1, per hour)

Size	XS	S	M	L	XL	2XL	3XL	4XL	5XL	6XL
Credits/hour	**1**	2	4	8	16	32	64	128	256	512

- Each size up **doubles** compute and credits. A query that runs twice as fast on the next size costs about the same.
- ⭐ **Per-second billing with a 60-second minimum** each time a warehouse **starts or resumes**. Resizing up also bills the added resources for at least 60 seconds.
- A **suspended warehouse uses no credits**.
- Multi-cluster cost = size credits × number of **running** clusters.

Worked examples

- XS runs for 30 minutes: 1 × 0.5 = **0.5 credits**.
- M resumes, runs one 10-second query, then suspends: billed 60 s, so 4 × 60/3600 = **0.067 credits**.
- L runs 2 hours with 3 clusters the whole time: 8 × 3 × 2 = **48 credits**.

Warehouse types

Type	What it is	Use for
Standard Gen1	Original warehouse hardware	General SQL
⭐ **Standard Gen2**	Faster hardware plus software optimisations for **DML (DELETE, UPDATE, MERGE) and table scans**. Set `GENERATION = '2'` (or `RESOURCE_CONSTRAINT = STANDARD_GEN_2`). **Default for new standard warehouses** where available. All sizes except 5XL and 6XL. Higher credit rate per hour, but most queries finish faster.	Analytics and data engineering
⭐ **Snowpark-optimized**	`WAREHOUSE_TYPE = 'SNOWPARK-OPTIMIZED'`. **16× memory per node** by default (`MEMORY_16X`). Options: `MEMORY_1X` (from XS), `MEMORY_16X` (from M), `MEMORY_64X` (from L, preview). Optional x86 CPU. Slower to start.	**ML training** in stored procedures, memory-heavy Snowpark UDFs/UDTFs

Type	What it is	Use for
Default warehouse for Notebooks	*Not tested until globally GA*	—

```
CREATE WAREHOUSE etl_wh
  WAREHOUSE_SIZE = 'MEDIUM'
  GENERATION = '2'
  AUTO_SUSPEND = 60             -- seconds idle before suspending (default 600)
  AUTO_RESUME = TRUE            -- default TRUE
  INITIALLY_SUSPENDED = TRUE
  MIN_CLUSTER_COUNT = 1
  MAX_CLUSTER_COUNT = 3         -- multi-cluster needs Enterprise+
  SCALING_POLICY = 'STANDARD';

ALTER WAREHOUSE etl_wh SET WAREHOUSE_SIZE = 'LARGE';   -- scale UP
ALTER WAREHOUSE etl_wh SUSPEND;
ALTER WAREHOUSE etl_wh RESUME;
```

⚠️ **Resizing only affects new or queued queries.** Queries already running finish on the old size.

⭐ Multi-cluster warehouses (Enterprise+) and scaling policies

- **Maximized mode:** `MIN_CLUSTER_COUNT = MAX_CLUSTER_COUNT > 1`. All clusters start with the warehouse. Use it for steady, high concurrency.
- **Auto-scale mode:** `MAX > MIN`. Clusters start and stop with load. **Scaling policy only applies in this mode.**
- Maximum clusters now depend on size: up to **300** for XS–M, 160 for L, 80 for XL, 40 for 2XL, 20 for 3XL, 10 for 4XL–6XL. Snowsight caps the field at 10. Older study material says "max 10".

Policy	Starts a new cluster...	Shuts a cluster down...	Choose it when
Standard (default)	As soon as a query **queues**, or the system estimates more capacity is needed	After sustained low load	**Performance and minimal queuing** matter
Economy	Only if there's enough load to keep it busy for **at least 6 minutes**	When less than about 6 minutes of work remains	**Saving credits**; some queuing is acceptable

⭐ Scale UP vs scale OUT

Problem	Fix	Why
One **complex or slow query**; **spilling** to local or remote storage	**Scale UP** (bigger size)	More memory, CPU and SSD per query
Many concurrent queries queuing	**Scale OUT** (multi-cluster)	More clusters share the queue
Paying for idle time	**AUTO_SUSPEND** low (e.g., 60 s for ETL and ad-hoc)	Credits stop when idle
BI slows down after every suspend	Longer AUTO_SUSPEND (e.g., 10 min)	Keeps the local SSD cache warm

Warehouse configuration by use case

Workload	Recommendation
Ad-hoc queries	Separate small/medium warehouse, short auto-suspend, scale up only if queries spill
Data loading	Dedicated warehouse; **XS–M** is usually enough. Load speed depends on the **number and size of files** more than warehouse size, because files load in parallel. A bigger size helps only when there are many files.
BI and reporting	**Multi-cluster auto-scale** for concurrency. Longer auto-suspend to keep the cache warm. Standard policy for latency, Economy for cost.
Different teams	**One warehouse per team or workload** for isolation and cost attribution (chargeback)
High concurrency	Scale **out** (multi-cluster). `MAX_CONCURRENCY_LEVEL` defaults to 8.
Complex queries	Scale **up**. Consider **QAS** for outlier queries, Snowpark-optimized for memory-hungry ML.

Other useful warehouse parameters: `STATEMENT_QUEUED_TIMEOUT_IN_SECONDS` (cancel queries that wait too long), `STATEMENT_TIMEOUT_IN_SECONDS`, `MAX_CONCURRENCY_LEVEL`, `ENABLE_QUERY_ACCELERATION`, `RESOURCE_MONITOR`.

⚠️ New **Gen2** warehouses and new **multi-cluster** warehouses have **QAS on by default with scale factor 2**. Converting an existing warehouse doesn't turn QAS on.

1.5 Explain Snowflake storage concepts

⭐ Micro-partitions

- Every table is split automatically into **micro-partitions** of **50–500 MB of uncompressed data** (much smaller once compressed).
- They are **columnar** inside, **immutable** (DML writes new partitions and the old ones move to Time Travel), and created **automatically in insertion order**. There's no manual partitioning.
- Metadata kept for each micro-partition: **min/max value range per column**, **number of distinct values**, **NULL count**, row count and more.
- ⭐ **Pruning:** the optimizer uses this metadata to skip partitions that can't match a filter. Compare "Partitions scanned" with "Partitions total" in the Query Profile to see how well pruning worked.
- Benefits: no partition maintenance, fine-grained pruning, column-level scanning, efficient compression.

⭐ Data clustering

- **Natural clustering:** data is clustered in the order it was loaded (often by date).
- Over time, DML scatters values, which increases overlap and worsens pruning.
- **Clustering depth** is the average number of overlapping micro-partitions for given columns. **Lower is better.** An empty table has depth 0; a well-clustered table is close to 1.
- Check it with `SYSTEM$CLUSTERING_INFORMATION('t', '(col)')`, which returns JSON with `average_depth`, `average_overlaps`, `total_constant_partition_count` and a depth histogram. `SYSTEM$CLUSTERING_DEPTH('t', '(col)')` returns just the depth.
- **Clustering keys** and **Automatic Clustering** (serverless) are covered in 4.2.

⭐ Table types

Type	Lives until	Time Travel	Fail-safe	Notes
Permanent (default)	Dropped	0–1 day (Standard) / **0–90** (Enterprise+)	**7 days**	Full protection, highest storage cost
Transient	Dropped	**0–1 day**	**None**	Cheaper for staging/ETL data you can rebuild. A transient **database or schema** makes all its tables transient.
Temporary	**End of session**	0–1 day	**None**	Visible only to the creating session. It can share a name with a permanent table and then **hides it** in that session.
External	Dropped (metadata only)	No	No	**Read-only** table over files in an **external stage**. Columns: `VALUE` (VARIANT) and `METADATA$FILENAME`. Refresh metadata manually or through cloud events. Slower than native tables; a materialized view on top helps.
⭐ **Apache Iceberg™**	Dropped	Snowflake-managed: yes	Only on Snowflake storage	Open table format, **Parquet** files in your **external volume** or Snowflake storage (see below)
⭐ **Dynamic**	Dropped	Yes	Yes (permanent)	Declarative pipeline: a query plus `TARGET_LAG`, refreshed by Snowflake (see 3.2)
Hybrid	Dropped	—	—	Unistore row store for **OLTP**: fast single-row lookups, **enforced primary keys**, indexes
Event	Dropped	—	—	Stores logs, traces and metrics

⚠️ You **can't** convert a permanent table to transient (or the reverse) with `ALTER`. Create a new table and copy the data.

⭐ Apache Iceberg tables

- Open format: data in **Parquet** plus Iceberg metadata. Other engines (Spark, Trino, Flink and others) can read and write the same data.
- **External volume:** an account-level object holding the IAM identity Snowflake uses to reach your S3, GCS or Azure storage. `EXTERNAL_VOLUME = 'SNOWFLAKE_MANAGED'` keeps the files in Snowflake storage instead.
- **Catalog options:**
  - **Snowflake as the catalog** (Snowflake-managed table): full read/write and full platform support (clustering, cloning, streams, replication). Snowflake handles maintenance such as compaction. External engines read it through **Snowflake Horizon Catalog's Iceberg REST API**.
  - **External catalog** through a **catalog integration**: AWS Glue, **Snowflake Open Catalog** (managed Apache Polaris), any Iceberg REST catalog, or metadata files in object storage. Platform support is limited, and writes work only through a REST catalog. A **catalog-linked database** can auto-discover and sync tables from a REST catalog.
- **Storage billing:** on your own external volume, **your cloud provider bills storage**, not Snowflake. Snowflake still bills compute.
- **Fail-safe applies only to permanent Iceberg tables on Snowflake storage**, never on customer-managed storage.
- An externally managed table can be **converted** to Snowflake-managed.

⭐ View types

View	Stores data?	Key facts
Standard	No	A named query. Definition visible to roles with privileges. Can join many tables.
Secure (`CREATE SECURE VIEW`)	No	**Definition hidden** from non-owners (`IS_SECURE` column). The optimizer skips rewrites that could expose hidden data, so it **can be slower**. Use for privacy and **sharing**.
Materialized (Enterprise+)	**Yes**	Pre-computed results **maintained automatically by a serverless service** (costs credits and storage). Always returns current data. **One table only, no joins.** Best for expensive, frequent queries on data that changes rarely (see 4.2).
Secure materialized	Yes	Both of the above

1.6 Explain AI/ML and application development features

Snowpark

- Libraries for **Python, Java and Scala** that build **DataFrame** pipelines. They're **lazily evaluated** and **pushed down** as SQL to run inside Snowflake warehouses. No data leaves Snowflake and no separate Spark cluster is needed.
- Write **UDFs, UDTFs, UDAFs and stored procedures** in Python, Java or Scala. Third-party Python packages come from the **Snowflake Anaconda channel** or a stage.
- **Snowpark pandas API:** pandas syntax at Snowflake scale.
- **Snowpark Container Services (SPCS):** run your own OCI/Docker containers on **compute pools**, with an image repository, services and jobs. Supports GPUs.
- Memory-heavy work belongs on a **Snowpark-optimized warehouse**.

Streamlit in Snowflake (SiS)

- Build and host **Python data apps** inside Snowflake. No separate hosting, and the data stays governed.
- Apps run with **owner's rights** (the app owner's role) and can be shared with other roles.
- They run on a warehouse (warehouse runtime) or on a compute pool (container runtime).
- Streamlit is also the UI layer for **Native Apps**.

Snowflake Notebooks

- Cell-based notebooks in Snowsight with **SQL, Python and Markdown** cells. Cells can use each other's results.
- **Warehouse Runtime:** fastest to start, Snowpark + Streamlit base, packages from Anaconda.
- **Container Runtime:** runs on a **compute pool** (SPCS) with CPU or **GPU** images and ML libraries; `pip` installs work. Built for ML.
- SQL and Snowpark queries always run on a **virtual warehouse**, whichever runtime is used.
- Supports **Git integration** and **scheduling** (runs as a task).
- Snowflake is moving to **Notebooks in Workspaces**; the original ones are now called "Legacy Notebooks". *The default notebook warehouse isn't tested yet.*

⭐ Snowflake Cortex (AI)

Fully managed AI that runs **inside Snowflake's security boundary**. Models (Snowflake Arctic, Mistral, Llama, Anthropic, OpenAI and others) are hosted by Snowflake. **Customer data isn't used to train the models.**

AI SQL functions (called in SQL like any other function):

Function	What it does
`AI_COMPLETE`	General LLM completion on text or images; the "do anything" function

Function	What it does
`AI_CLASSIFY`	Puts text or images into categories you list
`AI_FILTER`	Returns TRUE/FALSE for a natural-language condition, for `WHERE` or `JOIN`
`AI_AGG` / `AI_SUMMARIZE_AGG`	Insights or a summary **across many rows**, with no context-window limit
`AI_SENTIMENT`	Sentiment
`AI_EXTRACT`	Pulls fields or answers from text, images or documents
`AI_SUMMARIZE`	Summarises text, images or documents
`AI_TRANSLATE`	Translates between languages
`AI_EMBED` / `AI_MULTI_EMBED`	Embedding vectors (VECTOR data type)
`AI_SIMILARITY`	Similarity between two inputs
`AI_PARSE_DOCUMENT`	OCR or layout-aware text from staged PDFs and documents
`AI_TRANSCRIBE`	Speech-to-text for staged audio and video
`AI_REDACT`	Removes PII from text
`AI_COUNT_TOKENS`, `TO_FILE`, `PROMPT`	Helpers: count tokens, reference a staged file, build a prompt

Older names still appear in questions: `SNOWFLAKE.CORTEX.COMPLETE`, `SUMMARIZE`, `SENTIMENT`, `TRANSLATE`, `EXTRACT_ANSWER`, `CLASSIFY_TEXT`, `EMBED_TEXT_768/1024`, `PARSE_DOCUMENT`.

- **Access control:** the `SNOWFLAKE.CORTEX_USER` **database role** is **granted to PUBLIC by default**. To restrict access, revoke it from PUBLIC and grant it to chosen roles. Newer docs also mention the `USE AI FUNCTIONS` account privilege and the `AI_FUNCTIONS_USER` database role. An account-level model **allowlist** limits which models can be used, and **cross-region inference** can be enabled when a model isn't in your region.
- **Cost: credits per million tokens** (input tokens, plus output tokens for generation), plus the warehouse that runs the query. A bigger warehouse **doesn't** make LLM functions faster; Medium or smaller is recommended.
- The functions are tuned for **batch throughput**, not low latency. Interactive apps use the REST API.

Cortex Search ⭐

- Managed **hybrid search** (vector + keyword + semantic reranking) over text. It's the **retrieval engine for RAG** chatbots and app search bars, with no embeddings or infrastructure to manage.
- `CREATE CORTEX SEARCH SERVICE s ON <text_col> ATTRIBUTES <filter_cols> WAREHOUSE = w TARGET_LAG = '1 hour' AS (SELECT …);`
- Refreshes like a dynamic table (incrementally, when the source changes). Query it with `SEARCH_PREVIEW` (SQL test), the Python API or the REST API.
- Costs: warehouse for refreshes, embedding tokens, **serving cost per GB/month of indexed data**, storage.

Cortex Analyst ⭐

- **Natural language to SQL** over structured data, for business users. It's a **REST API**, usually called from a Streamlit app, a Slack/Teams bot or a Cortex Agent.
- Grounded in a **semantic model**: **semantic views** (recommended; schema-level objects with logical tables, dimensions, facts, metrics, relationships and verified queries) or legacy **YAML files** on a stage.
- Billed **per successful message**. The generated SQL runs on your warehouse and **respects RBAC**.

Related: Cortex Agents orchestrate tools (Analyst for structured data, Search for unstructured data, custom tools) for multi-step questions. **Snowflake Intelligence** is the business-user agent interface. **Document AI** extracts from documents, **Cortex Fine-tuning** customises models, **Cortex Guard** filters unsafe output and **Snowflake Copilot** assists with SQL.

⭐ Snowflake ML

- **Snowflake ML Python library** (`snowflake-ml-python`): feature engineering and training (scikit-learn, XGBoost, LightGBM, PyTorch) inside Snowflake, plus distributed training on the **Container Runtime** with CPUs or GPUs.
- **Feature Store:** define **entities** and **feature views** (backed by views or dynamic tables) so features are reused and kept fresh.
- **Model Registry:** log, version and govern models as **schema-level MODEL objects**, then run inference from **SQL or Python** on warehouses or SPCS.
- Also: ML Observability, Datasets, ML Jobs, experiment tracking.
- **ML Functions (SQL, no code):** `FORECAST` (time series), `ANOMALY_DETECTION`, `CLASSIFICATION`, `TOP_INSIGHTS` (what drives a metric). Example: `CREATE SNOWFLAKE.ML.FORECAST my_model(...)`.

✅ Domain 1: must remember

- Three layers: **Cloud Services** (the brain, result cache, metadata), **Compute** (warehouses), **Storage** (micro-partitions). Storage and compute scale independently.
- Cloud services are billed only above **10% of daily warehouse credits**.
- **Enterprise:** MV, SOS, QAS, multi-cluster, masking, row access, 90-day Time Travel, ACCESS_HISTORY. **Business Critical:** Tri-Secret Secure, PrivateLink, failover, HIPAA/PCI.
- Credits: XS = 1, doubling to 6XL = 512. **Per second, 60-second minimum** per start.
- **Scale up for complexity and spilling, out for concurrency.** Economy saves credits (6-minute rule); Standard minimises queuing.
- Gen2 = faster DML and scans, default for new warehouses. Snowpark-optimized = 16× memory for ML.
- Micro-partitions are **50–500 MB uncompressed**, immutable and columnar, with min/max metadata used for pruning.
- Transient and temporary tables: **no Fail-safe**, at most 1 day of Time Travel.
- Parameter precedence: **most specific wins** (session > user > account; table > schema > database > account).
- Stored procedures: **owner's rights by default**; `EXECUTE AS CALLER` uses the caller's privileges.
- Cortex: AI SQL functions billed by tokens; **Search = RAG / hybrid search**; **Analyst = text-to-SQL using semantic views**. `CORTEX_USER` is granted to PUBLIC by default.

Domain 2 — Account Management and Data Governance (20%)

2.1 Explain the Snowflake security model and principles

⭐ Access control models

- **DAC (discretionary access control):** every object has **one owner** (a role), and the owner can grant access to it.
- **RBAC (role-based access control):** privileges are granted to **roles**, and roles are granted to **users** (or to other roles). This is the main model.
- **UBAC (user-based access control):** privileges granted directly to users. They're considered only when `USE SECONDARY ROLES ALL` is active.
- Snowflake **combines DAC and RBAC**. Access is **denied unless a grant allows it**. There's **no super-user** that bypasses authorisation, not even ACCOUNTADMIN.

⭐ Securable object hierarchy

`Organization → Account → Database → Schema → schema objects`. A user needs privileges at **every level of the path**.

To query a table a role needs:

1. `USAGE` on the **database**
2. `USAGE` on the **schema**
3. `SELECT` on the **table** (or view)
4. `USAGE` on a **warehouse** to run the query

To create a table: `USAGE` on the database + `USAGE` and `CREATE TABLE` on the schema.

Warehouse privileges:

Privilege	Allows
`USAGE`	Run queries on it (and resume it if auto-resume is on)
`OPERATE`	Start, stop, suspend, resume; abort its queries
`MODIFY`	Change properties such as size; assign a resource monitor
`MONITOR`	See its queries and usage
`OWNERSHIP`	Full control

Useful account-level privileges: `CREATE DATABASE`, `CREATE WAREHOUSE`, `CREATE ROLE`, `CREATE USER`, `CREATE INTEGRATION`, `CREATE SHARE`, `IMPORT SHARE`, `MANAGE GRANTS`, `MONITOR USAGE`, `EXECUTE TASK`, `EXECUTE MANAGED TASK`, `EXECUTE ALERT`, `APPLY MASKING POLICY`, `APPLY ROW ACCESS POLICY`, `APPLY TAG`, `ATTACH POLICY`, `CREATE NETWORK POLICY`.

Grant syntax to recognise

```
GRANT USAGE ON DATABASE sales TO ROLE analyst;
GRANT USAGE ON SCHEMA sales.public TO ROLE analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA sales.public TO ROLE analyst;      -- existing tables only
GRANT SELECT ON FUTURE TABLES IN SCHEMA sales.public TO ROLE analyst;   -- tables created later
GRANT ROLE analyst TO ROLE sysadmin;        -- build the hierarchy
GRANT ROLE analyst TO USER ana;
GRANT OWNERSHIP ON TABLE sales.public.t TO ROLE etl COPY CURRENT GRANTS;
SHOW GRANTS TO ROLE analyst;    SHOW GRANTS ON TABLE t;    SHOW GRANTS OF ROLE analyst;
```

- ⚠️ `ON ALL …` covers **existing** objects only. `ON FUTURE …` covers **new** ones. You usually need both.
- ⚠️ When future grants exist at both database and schema level, the **schema-level ones win** for that schema.
- **Managed access schema** (`CREATE SCHEMA … WITH MANAGED ACCESS`): object owners can no longer grant privileges. Only the **schema owner** or a role with `MANAGE GRANTS` can. Use it to centralise grant decisions.

⭐ System-defined roles

```
              ACCOUNTADMIN
             /            \
      SECURITYADMIN      SYSADMIN
           |                |
        USERADMIN      custom roles (should roll up to SYSADMIN)
           \              /
                PUBLIC

  ORGADMIN / GLOBALORGADMIN sit outside this hierarchy (organization level)
```

Role	Purpose
GLOBALORGADMIN	Organization-level tasks (create/manage accounts, org usage). Exists only in the **organization account**; replacing ORGADMIN.
ORGADMIN	Organization tasks from a regular account: `CREATE ACCOUNT`, `SHOW ORGANIZATION ACCOUNTS`, enable replication, view `ORGANIZATION_USAGE`. Being phased out.
ACCOUNTADMIN	Top role. Inherits SYSADMIN + SECURITYADMIN. Billing, resource monitors, account parameters. **Grant to very few users, require MFA, don't make it a default role.**
SECURITYADMIN	Has `MANAGE GRANTS` (can grant/revoke any privilege). Inherits USERADMIN. Manages network policies.
USERADMIN	`CREATE USER` and `CREATE ROLE`. Manages the users and roles it owns.
SYSADMIN	Creates **warehouses, databases and other objects**. Custom roles should be granted to SYSADMIN so it can manage everything they create.
PUBLIC	Granted automatically to **every user and role**. Objects it owns are visible to everyone.

- System roles **can't be dropped**, and Snowflake's privileges on them can't be revoked.
- ⭐ A role that *owns* another role does **not** inherit that role's privileges. Inheritance only flows through **granting** one role to another.
- Best practice: create **custom roles** for job functions, and **grant them to SYSADMIN** so admins can manage their objects. Keep account-management privileges separate from object privileges.

⭐ Functional role types

Role type	Scope	Can be activated in a session?	Notes
Account role	Any object in the account	**Yes** (`USE ROLE`)	The normal role. Created by USERADMIN or any role with `CREATE ROLE`.
Database role	Objects in **one database**	⚠️ **No.** Grant it to an account role (or another database role in the same DB).	Created by the database owner. Can be **granted to a share** for fine-grained sharing. Moves with the database (cloning, replication).
Custom role	Any role you create (account or database)	—	Model job functions (ANALYST, ETL_ENGINEER…)
Application role	Inside a Native App	Granted to consumer roles	Defined in the app's setup script

Access roles vs functional roles pattern: create database/access roles that hold object privileges (e.g., `SALES_READ`, `SALES_WRITE`), then grant them to functional account roles (`ANALYST`, `ENGINEER`), which are granted to users.

⭐ Primary and secondary roles

- A session has exactly **one primary role** (`USE ROLE`). It's set from the connection, else the user's `DEFAULT_ROLE`, else PUBLIC.
- **Secondary roles** (`USE SECONDARY ROLES ALL | NONE | r1, r2`) add the privileges of other granted roles to the session. Users' `DEFAULT_SECONDARY_ROLES` activates them at login.
- ⭐ **Only the primary role authorises `CREATE` statements**, and the new object is **owned by the primary role**. Every other action (SELECT, INSERT, DDL on owned objects…) can be authorised by **any** active primary or secondary role.
- Use case: run a query that joins tables from two databases whose access lives in different roles, without building a parent role.
- `CURRENT_ROLE()` returns the primary role; `CURRENT_SECONDARY_ROLES()` returns the secondary ones; `IS_ROLE_IN_SESSION('R')` checks the whole active hierarchy.

⭐ Network policies

Restrict which IP addresses (or private endpoints) can connect.

```
CREATE NETWORK RULE corp_ips MODE = INGRESS TYPE = IPV4 VALUE_LIST = ('203.0.113.0/24');
CREATE NETWORK POLICY corp_only ALLOWED_NETWORK_RULE_LIST = ('corp_ips');
ALTER ACCOUNT SET NETWORK_POLICY = corp_only;          -- account level
ALTER USER etl_svc SET NETWORK_POLICY = etl_policy;     -- user level
```

- Older syntax: `ALLOWED_IP_LIST = ('…')`, `BLOCKED_IP_LIST = ('…')`. Network **rules** (types `IPV4`, `IPV6`, `AWSVPCEID`, `AZURELINKID`…) are the current approach.
- ⭐ **Precedence: most specific wins. Security integration > user > account.** A user-level policy overrides the account-level one.
- ⭐ If an IP is in **both** lists, the **blocked list wins**.
- **One active policy per account and per user** at a time.
- Created by **SECURITYADMIN or higher** (or a role with `CREATE NETWORK POLICY`). Activating at account level needs SECURITYADMIN+ or `ATTACH POLICY`.
- ⚠️ You can't activate an account policy that blocks **your own current IP**.
- Available in **all editions**. (Private connectivity itself is Business Critical.)

⭐ Authentication

Method	Key facts
Password + MFA	**MFA is required for human password users** (single-factor password sign-in is being retired). Methods: **passkeys (recommended), TOTP authenticator apps, Duo**. Users self-enrol. Admins can `ALTER USER … SET MINS_TO_BYPASS_MFA` or reset enrolment. Optional MFA token caching (up to 4 hours) for drivers.
Federated authentication / SSO	**SAML 2.0** identity provider (Okta, Microsoft Entra ID, ADFS, Ping…) configured with a **SAML2 security integration**. The IdP authenticates; Snowflake trusts it. Supports IdP- and Snowflake-initiated login. MFA is the IdP's job unless you enforce Snowflake MFA in an authentication policy.
OAuth	**Snowflake OAuth** (Snowflake is the authorisation server; used by partner apps like Tableau) or **External OAuth** (Okta, Entra ID, PingFederate issue tokens). Configured with an OAuth **security integration**. Clients connect with tokens, not passwords.
Key-pair	**RSA key pair, 2048-bit minimum.** Public key stored on the user (`ALTER USER u SET RSA_PUBLIC_KEY = '…'`); the client signs a JWT with the private key. `RSA_PUBLIC_KEY_2` allows **rotation without downtime**. Standard for **service accounts**, drivers, Kafka connector, CLI.
Programmatic access tokens (PATs)	Tokens used in place of a password by apps and service users; can be limited to a role and require a network policy
Workload identity federation	Service workloads on AWS, Azure, GCP or OIDC authenticate with their cloud identity, no secrets stored

- **User `TYPE`:** `PERSON` (humans), `SERVICE` (apps; **no password or MFA**; use key-pair, OAuth, PAT or workload identity), `LEGACY_SERVICE` (old password-based service users; being phased out).
- **Authentication policies** control allowed methods, client types, MFA enrolment and allowed security integrations, at account or user level.
- **SCIM** security integrations provision users and roles automatically from Okta or Entra ID.
- **Session policies** set idle timeouts; **password policies** set complexity, age and lockout.

⭐ Account identifiers

Format	Example	Notes
Organization + account name (preferred)	`myorg-analytics` → `https://myorg-analytics.snowflakecomputing.com`	Unique across clouds/regions; survives account moves
Account locator (legacy)	`xy12345.us-east-2.aws` → `https://xy12345.us-east-2.aws.snowflakecomputing.com`	Locator + region + cloud; `CURRENT_ACCOUNT()` returns it
Connection URL (client redirect)	`myorg-myconnection.snowflakecomputing.com`	Business Critical failover
Private connectivity	`…privatelink.snowflakecomputing.com`	Business Critical

In SQL, data sharing and replication use `orgname.account_name` (dot). URLs and connectors use `orgname-account_name` (hyphen).

Logging and tracing

- Code in **UDFs, stored procedures, Streamlit apps, Native Apps, SPCS services and notebooks** emits **logs, traces (spans with structured events) and metrics** in an **OpenTelemetry**-based format.
- Data lands in an **event table**. An active **default event table** (`SNOWFLAKE.TELEMETRY.EVENTS`) exists; you can create your own and set it with `ALTER ACCOUNT SET EVENT_TABLE = db.sch.my_events`.
- Control volume with **`LOG_LEVEL`** (e.g., `TRACE`, `DEBUG`, `INFO`, `WARN`, `ERROR`, `FATAL`, `OFF`), **`TRACE_LEVEL`** (`ALWAYS`, `ON_EVENT`, `OFF`) and **`METRIC_LEVEL`** (`ALL`, `NONE`). Set them on the account, database, schema, function/procedure or session. **The most specific (or most verbose, for logs) setting applies.**

- View with SQL on the event table or in **Snowsight → Monitoring → Traces & Logs**.
- Event tables are also used by **alerts**, the **Trust Center** and Native App event sharing.

2.2 Define data governance features and how they are used

Snowflake's governance suite is called **Snowflake Horizon** (catalog, security, privacy, lineage, discovery).

⭐ Column-level security: masking policies (Enterprise+)

```
CREATE MASKING POLICY email_mask AS (val STRING) RETURNS STRING ->
  CASE WHEN IS_ROLE_IN_SESSION('PII_READER') THEN val
       ELSE REGEXP_REPLACE(val, '.+@', '*****@') END;

ALTER TABLE customers MODIFY COLUMN email SET MASKING POLICY email_mask;
```

- **Dynamic data masking:** applied **at query time**, the stored data is unchanged. The same policy can protect many columns.
- Input and return data types **must match** the column type. **One masking policy per column.**
- **Conditional masking:** `AS (val STRING, region STRING) + USING (email, region)` masks based on another column.
- **Tag-based masking:** attach the policy to a **tag**; every column with that tag is masked automatically (also at database/schema level).
- **External tokenization:** data is tokenised before loading; the policy calls an **external function** to detokenise for authorised roles.
- Prefer `IS_ROLE_IN_SESSION()` (respects role hierarchy and secondary roles) over `CURRENT_ROLE()` (exact match only). `INVOKER_ROLE()` helps with views.
- Requires `APPLY MASKING POLICY` (account) or `APPLY` on the policy + ownership of the table.

⭐ Row-level security: row access policies (Enterprise+)

```
CREATE ROW ACCESS POLICY region_rap AS (region STRING) RETURNS BOOLEAN ->
  IS_ROLE_IN_SESSION('SALES_ADMIN')
  OR EXISTS (SELECT 1 FROM gov.region_map m
             WHERE m.role_name = CURRENT_ROLE() AND m.region = region);

ALTER TABLE sales ADD ROW ACCESS POLICY region_rap ON (region);
```

- Returns **BOOLEAN**: TRUE = row visible. Often uses a **mapping table**.
- **One row access policy per table or view.**
- ⭐ **Evaluation order: row access policy first, then masking policies.**
- For shared data, use `CURRENT_ACCOUNT()` or database-role checks so each consumer sees only its rows.

⭐ Object tagging

```
CREATE TAG cost_center ALLOWED_VALUES 'finance', 'engineering';
ALTER WAREHOUSE bi_wh SET TAG cost_center = 'finance';
ALTER TABLE customers MODIFY COLUMN ssn SET TAG pii = 'ssn';
SELECT SYSTEM$GET_TAG('cost_center', 'bi_wh', 'warehouse');
```

- Tags are **schema-level objects** with a string value; they can be set on accounts, warehouses, databases, schemas, tables, columns, users, roles and more.
- **Inheritance:** a tag on a database or schema is inherited by the objects inside it. **Tag propagation** can carry tags to derived objects automatically.
- Uses: **cost attribution** (tag warehouses/queries), **tag-based masking**, **classification**, discovery.
- Views: `ACCOUNT_USAGE.TAG_REFERENCES`, the `TAG_REFERENCES` table function, `POLICY_REFERENCES`.
- Tagging works in all editions; tag-based masking needs Enterprise.

Data classification (Enterprise+)

- Automatically detects **sensitive data** (email, name, phone, national IDs…) and assigns system tags `SNOWFLAKE.CORE.SEMANTIC_CATEGORY` and `SNOWFLAKE.CORE.PRIVACY_CATEGORY` (`IDENTIFIER`, `QUASI_IDENTIFIER`, `SENSITIVE`).
- Can run automatically on a schedule using **classification profiles**, and can apply tags (and therefore tag-based masking) automatically.

⭐ Privacy policies (Enterprise+)

Policy	What it enforces	Typical use
Aggregation policy	Queries **must aggregate**, and every group must contain at least a **minimum group size** of rows. Small groups are folded into a remainder group.	Prevent re-identification of individuals; clean rooms
Projection policy	A column **can't appear in the SELECT output**, but can still be used in `WHERE` or `JOIN`	Join on email without revealing email
Join policy	Requires the table to be joined, optionally on specific columns	Clean rooms
Differential privacy (privacy policy + privacy domains)	Adds calibrated **noise** to aggregate results and tracks a **privacy budget** per consumer	Strong mathematical privacy guarantees
Synthetic data	`GENERATE_SYNTHETIC_DATA` creates statistically similar fake data	Testing and sharing

Row access policies take precedence over privacy policies. These policies are the building blocks of **data clean rooms**.

⭐ Trust Center

- Evaluates the account's **security posture** against recommendations using **scanners** grouped into **scanner packages**, and shows **findings** with severity (low, medium, high, critical) and remediation steps.
- Packages: **Security Essentials** (always on, **free**, checks MFA for password users, account network policy, event table setup), **CIS Benchmarks** (off by default), **Threat Intelligence** (off by default; risky users, dormant users, unusual IPs, auth failures…), plus newer AI Security and Data Security packages.
- **Findings:** *violations* (persistent configurations to fix) and *detections* (one-time events to investigate).
- ⭐ **Cost:** scanners other than the scheduled Security Essentials run use **serverless compute** (billed).
- Access: ACCOUNTADMIN, or roles granted the **`SNOWFLAKE.TRUST_CENTER_ADMIN`** / **`TRUST_CENTER_VIEWER`** application roles. Email and webhook notifications for findings.

⭐ Encryption and key management

- **All data is encrypted at rest (AES-256) and in transit (TLS 1.2+)**, in **every edition**, with no setup.
- **Hierarchical key model:** root key (in a hardware security module) → **account master keys** → **table master keys** → **file keys**. Each key encrypts only the keys below it, which limits exposure.
- ⭐ **Automatic key rotation:** active keys are **rotated every 30 days**; retired keys only decrypt; destroyed keys are gone.

- ⭐ **Periodic rekeying (Enterprise+):** data encrypted with a key **older than 1 year** is **re-encrypted** with a new key (`PERIODIC_DATA_REKEYING = TRUE`).
- ⭐ **Tri-Secret Secure (Business Critical+):** a **composite master key** combines a Snowflake-managed key with a **customer-managed key** in AWS KMS, Azure Key Vault or Google Cloud KMS. **Revoke your key and Snowflake can no longer decrypt your data.**
- Internal stages: client-side + server-side encryption by default (`SNOWFLAKE_FULL`). See 3.1.

⭐ Alerts

Run a condition on a schedule and act when it's met.

```
CREATE ALERT big_orders_alert
  WAREHOUSE = alert_wh                      -- omit for a serverless alert
  SCHEDULE = '60 MINUTE'
  IF (EXISTS (
        SELECT 1 FROM orders
        WHERE amount > 1000000
          AND created_at > SNOWFLAKE.ALERT.LAST_SUCCESSFUL_SCHEDULED_TIME()))
  THEN
    CALL SYSTEM$SEND_EMAIL('email_int', 'ops@example.com', 'Big order', 'Check orders');

ALTER ALERT big_orders_alert RESUME;          -- alerts are created SUSPENDED
```

- Needs `EXECUTE ALERT` (account) and, for serverless alerts, `EXECUTE MANAGED ALERT`.
- Can also run **on new data** (event-driven) instead of a schedule.
- History in `ALERT_HISTORY` (table function and Account Usage view).

⭐ Notifications

- Delivered through **notification integrations**:
  - `TYPE = EMAIL`: send to **verified** email addresses of account users (`SYSTEM$SEND_EMAIL`).
  - `TYPE = WEBHOOK`: Slack, Microsoft Teams, PagerDuty.
  - `TYPE = QUEUE`: **AWS SNS, Azure Event Grid, Google Pub/Sub** (outbound error notifications, and inbound events for Snowpipe auto-ingest on Azure/GCP).
- `SYSTEM$SEND_SNOWFLAKE_NOTIFICATION` sends to any integration type.
- Error notifications for **tasks and pipes** use the `ERROR_INTEGRATION` parameter.
- Resource monitors and budgets send their own email notifications.

Data replication and failover

Covered in detail in **5.1**. Key governance point: **failover requires Business Critical**; database replication works in all editions.

⭐ Data lineage

- **Snowsight lineage graph:** shows upstream sources and downstream objects (tables, views, dynamic tables) and **column-level lineage**, including how data moved (CTAS, INSERT…SELECT, MERGE, COPY).
- **`ACCOUNT_USAGE.ACCESS_HISTORY` (Enterprise+):** who read or wrote which **objects and columns**, with **direct** vs **base** objects (base = the underlying tables behind a view). Used for audits, unused-data analysis and lineage. Latency up to about 3 hours.
- **`OBJECT_DEPENDENCIES`** view: which objects depend on which (e.g., view → table).
- Lineage also helps **propagate tags** and assess the impact of a change.

2.3 Explain monitoring and cost management

⭐ What costs money

Category	Includes
Compute	Virtual warehouses (including dynamic-table refreshes, which run on your warehouse); **serverless** features (Snowpipe, Automatic Clustering, MV maintenance, SOS, QAS, serverless tasks and alerts, replication); **cloud services** (above 10% of daily warehouse usage); AI services (Cortex tokens)
Storage	Average compressed TB/month: active tables, **Time Travel**, **Fail-safe**, internal stages, clones' changed data
Data transfer	**Egress** to another region or cloud (replication, unloading, external functions). Ingress is free.

⭐ Resource monitors

```
CREATE RESOURCE MONITOR monthly_bi WITH
  CREDIT_QUOTA = 500
  FREQUENCY = MONTHLY
  START_TIMESTAMP = IMMEDIATELY
  TRIGGERS ON 75 PERCENT DO NOTIFY
           ON 90 PERCENT DO SUSPEND
           ON 100 PERCENT DO SUSPEND_IMMEDIATE;

ALTER WAREHOUSE bi_wh SET RESOURCE_MONITOR = monthly_bi;   -- warehouse-level
ALTER ACCOUNT SET RESOURCE_MONITOR = account_cap;           -- account-level
```

- ⭐ They control **user-managed warehouses only** (plus the cloud services that support them). They **can't** control **serverless features or AI services**. Use **budgets** for those.
- **Actions:** `NOTIFY`; `SUSPEND` (suspends **after running queries finish**); `SUSPEND_IMMEDIATE` (**cancels** running queries). Limits: **up to 5 NOTIFY, 1 SUSPEND, 1 SUSPEND_IMMEDIATE**. Thresholds can exceed 100%.
- **One account-level monitor** per account. A **warehouse can be assigned to only one** warehouse-level monitor, but one monitor can cover **many warehouses**.
- **Frequency:** `DAILY` | `WEEKLY` | `MONTHLY` | `YEARLY` | `NEVER`. Default: monthly, starting immediately. **Quotas reset at 12:00 AM UTC.**
- ⭐ **Only ACCOUNTADMIN** can create monitors (other roles can get `MONITOR` or `MODIFY` on a specific one).
- Suspended warehouses stay suspended until the next interval, or until the quota or threshold is raised or the warehouse is removed from the monitor.
- Monitors aren't exact; a warehouse can run briefly past the threshold. Leave a buffer (e.g., suspend at 90%).

Budgets

- Track **spend including serverless features and AI services**, for the **whole account** (account budget) or a **custom group of objects** (custom budgets, can use tags).
- Set a **monthly spending limit** and get notified (email, webhook, queue) when spend is **projected** to exceed it. Budgets notify; they don't suspend warehouses the way resource monitors do.

⭐ Calculating virtual warehouse credit usage

- Credits = **credits/hour for the size × hours running × running clusters**, billed **per second** with a **60-second minimum** per start/resume.
- **Cloud services:** billed only for the portion above **10% of daily warehouse credits**.
- **Serverless features** have their own credit multipliers (Service Consumption Table).

- Monitor with `WAREHOUSE_METERING_HISTORY` (per warehouse per hour), `METERING_HISTORY`, `METERING_DAILY_HISTORY` (includes the cloud services adjustment), and **Snowsight → Admin → Cost Management**.

⭐ ACCOUNT_USAGE vs INFORMATION_SCHEMA

	`SNOWFLAKE.ACCOUNT_USAGE`	`<db>.INFORMATION_SCHEMA`
Where	Shared `SNOWFLAKE` database	Every database (views + table functions)
Scope	Whole account	That database (plus some account objects)
Dropped objects	**Included**	Not included
Latency	**45 minutes to 3 hours** (varies by view)	**None** (real-time)
Retention	**365 days (1 year)**	**7 days to 6 months** (e.g., `QUERY_HISTORY()` 7 days, `COPY_HISTORY` 14 days)
Access	**ACCOUNTADMIN by default**; grant `IMPORTED PRIVILEGES` on the SNOWFLAKE database or the SNOWFLAKE database roles `OBJECT_VIEWER`, `USAGE_VIEWER`, `GOVERNANCE_VIEWER`, `SECURITY_VIEWER`	Any role, filtered by its privileges

Key ACCOUNT_USAGE views to know:

View	Use
`QUERY_HISTORY`	Every query: times, queued time, bytes scanned, spilling, partitions, warehouse, role, `QUERY_TAG`
`QUERY_ATTRIBUTION_HISTORY`	⭐ **Compute credits per query** (see 4.1)
`WAREHOUSE_METERING_HISTORY`	Credits per warehouse per hour
`METERING_HISTORY` / `METERING_DAILY_HISTORY`	Credits by service type (warehouses, serverless, AI), daily cloud services adjustment
`WAREHOUSE_LOAD_HISTORY`	Running vs queued load per warehouse
`STORAGE_USAGE`, `DATABASE_STORAGE_USAGE_HISTORY`, `TABLE_STORAGE_METRICS`	Storage incl. active, Time Travel, Fail-safe bytes
`LOGIN_HISTORY`, `SESSIONS`	Authentication attempts, client types
`ACCESS_HISTORY`	Who read/wrote which columns (Enterprise+)
`COPY_HISTORY`	Bulk COPY **and Snowpipe** loads
`LOAD_HISTORY`	Bulk COPY only (⚠️ **not** Snowpipe)
`PIPE_USAGE_HISTORY`, `AUTOMATIC_CLUSTERING_HISTORY`, `MATERIALIZED_VIEW_REFRESH_HISTORY`, `SEARCH_OPTIMIZATION_HISTORY`, `QUERY_ACCELERATION_HISTORY`, `SERVERLESS_TASK_HISTORY`	Serverless feature costs
`TASK_HISTORY`, `ALERT_HISTORY`	Task and alert runs
`USERS`, `ROLES`, `GRANTS_TO_ROLES`, `GRANTS_TO_USERS`	Security audits
`TAG_REFERENCES`, `POLICY_REFERENCES`	Governance

View	Use
`DATA_TRANSFER_HISTORY`, `REPLICATION_GROUP_USAGE_HISTORY`	Egress and replication costs
`QUERY_INSIGHTS`	Performance insights (see 4.1)

Also: **`ORGANIZATION_USAGE`** (across all accounts in the organization) and **`READER_ACCOUNT_USAGE`** (usage of reader accounts).

✅ Domain 2: must remember

- **DAC + RBAC**. No super-user. Querying needs USAGE on DB + schema, SELECT on the object, USAGE on a warehouse.
- **SECURITYADMIN = MANAGE GRANTS**; **USERADMIN = users and roles**; **SYSADMIN = objects**; ACCOUNTADMIN = top; PUBLIC = everyone.
- **Database roles can't be activated directly**; grant them to account roles. They can be granted to shares.
- **Only the primary role can CREATE**; secondary roles authorise everything else.
- Network policy precedence: **integration > user > account**; **blocked list beats allowed list**.
- Key-pair: **RSA 2048+**, `RSA_PUBLIC_KEY_2` for rotation. SERVICE users have no password/MFA.
- **Row access policy runs before masking.** One masking policy per column; one row access policy per table.
- Keys rotate **every 30 days**; **rekeying yearly** (Enterprise); **Tri-Secret Secure** = Business Critical.
- Resource monitors: **warehouses only**, ACCOUNTADMIN creates, 5 NOTIFY / 1 SUSPEND / 1 SUSPEND_IMMEDIATE, reset at midnight UTC. **Budgets** cover serverless and AI.
- ACCOUNT_USAGE: **1 year, has dropped objects, 45 min–3 h latency.** INFORMATION_SCHEMA: **real-time, 7 days–6 months, no dropped objects.**
- `LOAD_HISTORY` excludes Snowpipe; `COPY_HISTORY` includes it.

Domain 3 — Data Loading, Unloading, and Connectivity (18%)

3.1 Perform data loading and unloading

⭐ File formats

Format	Load	Unload	Notes
CSV / delimited	✅	✅	Default type
JSON	✅	✅	Loads into VARIANT; unload needs VARIANT/OBJECT data
Parquet	✅	✅	Columnar; also used by Iceberg
Avro	✅	❌	
ORC	✅	❌	
XML	✅	❌	

⭐ Unload formats are only CSV, JSON and Parquet.

```
CREATE FILE FORMAT my_csv
  TYPE = CSV
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1                      -- or PARSE_HEADER = TRUE for MATCH_BY_COLUMN_NAME
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  NULL_IF = ('NULL', '')
  EMPTY_FIELD_AS_NULL = TRUE
  ERROR_ON_COLUMN_COUNT_MISMATCH = TRUE
  COMPRESSION = AUTO;                  -- auto-detects gzip, bz2, zstd, brotli…

CREATE FILE FORMAT my_json TYPE = JSON STRIP_OUTER_ARRAY = TRUE STRIP_NULL_VALUES = FALSE;
```

- ⭐ `STRIP_OUTER_ARRAY = TRUE` loads each element of a top-level JSON array as **its own row** (otherwise the whole array is one row and can hit the VARIANT size limit).
- **Option precedence:** options in the **COPY statement** override the **stage's** file format, which overrides the **table's** stage file format.
- Copy options (ON_ERROR, PURGE…) belong in `COPY INTO`, not in `CREATE STAGE`.

⭐ Stages

Stage	Reference	Created	Key facts
User stage	`@~`	Automatically, one per user	Only that user can access it. Can't be altered or dropped. Good for one user's files.
Table stage	`@%my_table`	Automatically, one per table	Many users, **one table only**. Can't be altered or dropped; not a separate object, so no grants (needs table ownership). ⚠️ **No transformations** during COPY from it.
Named internal stage	`@my_stage`	`CREATE STAGE`	Most flexible: grants, file format, directory table, used by many users/tables

Stage	Reference	Created	Key facts
Named external stage	`@my_ext_stage`	`CREATE STAGE … URL = 's3://…'`	Points at **S3, GCS or Azure Blob** (also S3-compatible). Use a **storage integration**. Dropping it doesn't delete files.

```
CREATE STAGE raw_ext
  URL = 's3://my-bucket/raw/'
  STORAGE_INTEGRATION = s3_int           -- preferred over CREDENTIALS = (...)
  FILE_FORMAT = my_csv
  DIRECTORY = (ENABLE = TRUE AUTO_REFRESH = TRUE);

-- From a client (Snowflake CLI, SnowSQL, driver) - NOT from a Snowsight worksheet:
PUT file:///tmp/orders_*.csv @my_stage/orders/ AUTO_COMPRESS = TRUE PARALLEL = 8;
GET @my_stage/exports/ file:///tmp/exports/;

LIST @my_stage/orders/;
REMOVE @my_stage/orders/ PATTERN = '.*2025.*';
```

- ⭐ `PUT` **uploads to internal stages only**; `GET` **downloads from internal stages only**. For external stages, use the cloud provider's tools.
- `PUT` **gzip-compresses by default** (`AUTO_COMPRESS = TRUE`), encrypts files, skips files that already exist unless `OVERWRITE = TRUE`, and uploads in parallel (`PARALLEL`, default 4).
- You can **query staged files directly** without loading:

```
SELECT $1, $2, METADATA$FILENAME, METADATA$FILE_ROW_NUMBER, METADATA$FILE_LAST_MODIFIED
FROM @my_stage/orders/ (FILE_FORMAT => 'my_csv', PATTERN => '.*[.]csv[.]gz');
```

⭐ Server-side encryption (stages)

Stage	Option	Meaning
Internal	`ENCRYPTION = (TYPE = 'SNOWFLAKE_FULL')` **default**	Client-side + server-side encryption. Required for **Tri-Secret Secure**.
Internal	`ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE')`	**Server-side only.** ⭐ Needed so staged files can be read through **scoped, file or pre-signed URLs** (unstructured data).
S3	`AWS_SSE_S3`, `AWS_SSE_KMS` (optional `KMS_KEY_ID`), `AWS_CSE` (client-side with `MASTER_KEY`), `NONE`	
GCS	`GCS_SSE_KMS`, `NONE`	
Azure	`AZURE_CSE`, `NONE`	

⚠️ The **encryption type can't be changed** after the stage is created.

⭐ Directory tables

- An implicit catalogue of **files in a stage** (path, size, last modified, MD5, `FILE_URL`). Enable with `DIRECTORY = (ENABLE = TRUE)`.
- Refresh **manually** (`ALTER STAGE s REFRESH`) or **automatically** (`AUTO_REFRESH = TRUE` with cloud event notifications for external stages).
- Query with `SELECT * FROM DIRECTORY(@s);`. You can put a **stream** on a directory table to process new files with a task.

- Foundation for **unstructured data** (see 4.4).

⭐ COPY INTO a table

```
COPY INTO sales.raw.orders
FROM @raw_ext/orders/
FILE_FORMAT = (FORMAT_NAME = 'my_csv')
PATTERN = '.*orders_.*[.]csv[.]gz'     -- regex; or FILES = ('a.csv','b.csv') (max 1,000)
ON_ERROR = 'SKIP_FILE'
PURGE = TRUE;
```

Load metadata ⭐

- Snowflake remembers which files were loaded into each table for **64 days** (name + ETag/checksum). Re-running COPY **skips files already loaded**, so it's idempotent.
- `FORCE = TRUE` reloads **all** files regardless (can create **duplicates**).
- Files whose last-modified date is **older than 64 days** and aren't in the metadata have an **uncertain** load status and are skipped. `LOAD_UNCERTAIN_FILES = TRUE` loads them.
- ⭐ `TRUNCATE TABLE` **clears the load metadata** (the same files can be loaded again). `DELETE` doesn't.

Key COPY options

Option	Meaning
`ON_ERROR`	⭐ `ABORT_STATEMENT` (**default for bulk COPY**), `CONTINUE` (skip bad rows), `SKIP_FILE` (**default for Snowpipe**), `SKIP_FILE_<n>`, `'SKIP_FILE_<n>%'`
`VALIDATION_MODE`	⭐ `RETURN_<n>_ROWS`, `RETURN_ERRORS`, `RETURN_ALL_ERRORS`. **Validates without loading.** Not supported with COPY transformations.
`PURGE`	⭐ `TRUE` deletes files from the stage **after a successful load** (default FALSE). A purge failure doesn't fail the load.
`FORCE`	Reload files even if already loaded
`LOAD_UNCERTAIN_FILES`	Load files with unknown status (older than 64 days)
`SIZE_LIMIT`	Stop after roughly this many bytes (at least one file is loaded)
`MATCH_BY_COLUMN_NAME`	`CASE_SENSITIVE` / `CASE_INSENSITIVE` / `NONE`: map Parquet/Avro/ORC/JSON (and CSV with `PARSE_HEADER`) fields to columns by name
`TRUNCATECOLUMNS` / `ENFORCE_LENGTH`	Truncate strings that are too long instead of erroring
`RETURN_FAILED_ONLY`	Output only files that failed
`INCLUDE_METADATA`	Load `METADATA$FILENAME` etc. into columns (with MATCH_BY_COLUMN_NAME)

- **After a load**, see errors with `SELECT * FROM TABLE(VALIDATE(orders, JOB_ID => '_last'));`.
- Monitor with `COPY_HISTORY` (table function and Account Usage view), `LOAD_HISTORY`, and Snowsight Copy History.

Simple transformations during COPY ⭐

```
COPY INTO orders (order_id, customer, amount, loaded_at, src_file)
FROM (SELECT t.$1, UPPER(t.$3), t.$2::NUMBER(10,2), CURRENT_TIMESTAMP(), METADATA$FILENAME
      FROM @raw_ext/orders/ t)
FILE_FORMAT = (FORMAT_NAME = 'my_csv');
```

- Supported: **reorder columns, omit columns, cast types, truncate, simple functions**, add sequences/defaults, file metadata.
- ⚠️ **Not supported:** `WHERE` filters, `JOIN`, `GROUP BY`, `ORDER BY`, aggregates, `FLATTEN`. Load first, then transform with SQL.

⭐ Loading best practices

- **File size: 100–250 MB compressed** per file. Split huge files; combine tiny ones (per-file overhead).
- Files load **in parallel** across warehouse threads, so many right-sized files load faster than one huge file.
- Organise stage paths by date/source (`/orders/2026/10/24/`) and load with path prefixes or `PATTERN`.
- Use a **dedicated loading warehouse**.
- **Semi-structured:** load into a **VARIANT** column; use `STRIP_OUTER_ARRAY` for arrays.
- **Schema detection:** `INFER_SCHEMA()` + `CREATE TABLE … USING TEMPLATE` builds a table from Parquet/Avro/ORC/JSON/CSV files. **Schema evolution** (`ENABLE_SCHEMA_EVOLUTION = TRUE`) adds new columns automatically during loads.
- Snowsight's **load wizard** handles small files (up to 250 MB each).

⭐ Unloading (COPY INTO a stage or location)

```
COPY INTO @my_stage/exports/orders_
FROM (SELECT * FROM orders WHERE order_date >= '2026-01-01')
FILE_FORMAT = (TYPE = PARQUET)
HEADER = TRUE                -- keep column names (CSV/Parquet)
PARTITION BY ('date=' || TO_VARCHAR(order_date))
MAX_FILE_SIZE = 104857600    -- bytes; default 16 MB
OVERWRITE = TRUE;
-- then GET @my_stage/exports/ file:///local/  (internal stages)
```

- Default output: **multiple files in parallel** (`SINGLE = FALSE`) of up to **16 MB** each (`MAX_FILE_SIZE`). `SINGLE = TRUE` writes one file (max **5 GB** to cloud storage).
- CSV/JSON output is **gzip-compressed** by default; Parquet uses Snappy.
- Can unload **directly to an external location** (`COPY INTO 's3://…' STORAGE_INTEGRATION = …`).
- JSON unload: build objects with `OBJECT_CONSTRUCT(*)` so each row becomes a JSON object.
- Useful options: `HEADER`, `OVERWRITE`, `PARTITION BY`, `INCLUDE_QUERY_ID`, `DETAILED_OUTPUT`, `FILE_EXTENSION`.

3.2 Perform automated data ingestion

⭐ Snowpipe (continuous, file-based)

```
CREATE PIPE raw.orders_pipe
  AUTO_INGEST = TRUE
  ERROR_INTEGRATION = pipe_errors_int          -- optional error notifications
AS
  COPY INTO raw.orders FROM @raw_ext/orders/ FILE_FORMAT = (FORMAT_NAME = 'my_json');

SHOW PIPES;                                     -- notification_channel = SQS ARN for S3
SELECT SYSTEM$PIPE_STATUS('raw.orders_pipe');
ALTER PIPE raw.orders_pipe REFRESH;             -- queue files staged in the last 7 days
ALTER PIPE raw.orders_pipe SET PIPE_EXECUTION_PAUSED = TRUE;
```

- A **pipe** = a `COPY INTO` statement Snowflake runs **automatically when new files arrive**, on **serverless compute** (no warehouse).
- **Two triggers:**
  1. **Auto-ingest** with **cloud event notifications**: S3 → **SQS** (or SNS), Azure → **Event Grid** (+ notification integration), GCS → **Pub/Sub** (+ notification integration). External stages.
  2. **REST API** (`insertFiles` endpoint) called by your app, using **key-pair auth**. Works for internal and external stages.
- ⭐ **Load history kept 14 days** in pipe metadata (vs 64 days for bulk COPY). Prevents reloading the same file.
- ⭐ **Default `ON_ERROR` is `SKIP_FILE`.** `PURGE` isn't supported; clean up with `REMOVE` or storage lifecycle rules.
- Typical latency is about a minute. **Load order isn't guaranteed.**
- ⭐ **Billing:** a fixed **credits per GB** loaded (0.0037 credits/GB; text files counted uncompressed), **no per-file charge** (since late 2025; older material mentions compute time + a per-1,000-files fee). Monitor with `PIPE_USAGE_HISTORY`.
- ⚠️ `CREATE OR REPLACE PIPE` **resets the load history**, which can cause duplicate loads.

⭐ Snowpipe Streaming (continuous, row-based)

- Applications send **rows** (not files) straight into tables through the **Snowpipe Streaming SDK** (Java, Python, Node.js) or REST API. **No staging files.**
- **Lowest latency**: data queryable in about **5–10 seconds**.
- **Channels:** connections that carry ordered rows. **Offset tokens** track progress for **exactly-once** recovery (named channels). Newer **elastic channels** scale automatically.
- **High-performance architecture** (current): uses a **PIPE object** for transformations, **throughput-based billing (credits per uncompressed GB)**. The classic architecture used the Java Ingest SDK without a pipe.
- The **Snowflake Connector for Kafka** can use Snowpipe Streaming (recommended) or Snowpipe.
- Can write to standard tables and Snowflake-managed **Iceberg** tables.

Choosing an ingestion method

Method	Unit	Compute	Latency	History	Default ON_ERROR	Best for
COPY INTO	Files (batch)	**Your warehouse**	Scheduled batches	**64 days**	**ABORT_STATEMENT**	Large scheduled bulk loads
Snowpipe	Files (micro-batch)	**Serverless**	~1 minute	**14 days**	**SKIP_FILE**	Files arriving continuously in cloud storage
Snowpipe Streaming	**Rows**	Serverless	**Seconds**	Offset tokens	—	Kafka, IoT, app events, CDC

⭐ Streams (change data capture)

```
CREATE STREAM orders_stream ON TABLE raw.orders;                    -- standard (delta)
CREATE STREAM orders_ins ON TABLE raw.orders APPEND_ONLY = TRUE;     -- inserts only
CREATE STREAM ext_new ON EXTERNAL TABLE ext.events INSERT_ONLY = TRUE;

SELECT * FROM orders_stream;          -- querying does NOT advance the offset
INSERT INTO curated.orders SELECT … FROM orders_stream WHERE METADATA$ACTION = 'INSERT';
                                      -- consuming in DML DOES advance it (on commit)
```

- A stream stores **only an offset** (a point in the table's version history), not data. It returns the rows changed **since the offset**.

- **Metadata columns:** `METADATA$ACTION` (`INSERT` / `DELETE`), `METADATA$ISUPDATE` (TRUE for the delete+insert pair of an UPDATE), `METADATA$ROW_ID`.
- ⭐ **The offset advances only when the stream is consumed in a committed DML transaction** (INSERT, MERGE, CTAS, COPY INTO location…). A plain `SELECT` doesn't advance it.
- **Repeatable read:** every statement in one transaction sees the same stream contents.

Stream type	Tracks	Source objects
Standard (delta)	Inserts, updates, deletes (net change)	Tables, views (incl. secure views), dynamic tables, directory tables, Snowflake-managed Iceberg
Append-only	**Inserts only** (cheaper, faster)	Standard tables, dynamic tables, Snowflake-managed Iceberg, views
Insert-only	Inserts only (new files)	**External tables**, externally managed Iceberg tables

- ⭐ **Staleness:** a stream becomes stale if not consumed within the table's retention period. Snowflake **extends retention up to `MAX_DATA_EXTENSION_TIME_IN_DAYS` (default 14 days)** for unconsumed streams. Check `STALE_AFTER` in `SHOW STREAMS`. A stale stream must be recreated.
- `SYSTEM$STREAM_HAS_DATA('s')` tells a task whether to run.
- **One stream per consumer.** Two consumers of one stream interfere with each other.
- ⚠️ **Cloned streams don't carry unconsumed records** (official sample Q2). `CREATE OR REPLACE TABLE` makes its streams stale.
- `CHANGES` clause: query change tracking between two points without a stream (`SELECT … FROM t CHANGES(INFORMATION => DEFAULT) AT(…)`), needs change tracking enabled.

⭐ Tasks

```
CREATE TASK load_curated
  WAREHOUSE = etl_wh                          -- omit for SERVERLESS
  SCHEDULE = 'USING CRON 0 2 * * * UTC'      -- or '15 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('orders_stream')
AS
  MERGE INTO curated.orders t USING orders_stream s ON t.id = s.id
  WHEN MATCHED THEN UPDATE SET t.amount = s.amount
  WHEN NOT MATCHED THEN INSERT (id, amount) VALUES (s.id, s.amount);

ALTER TASK load_curated RESUME;               -- tasks are created SUSPENDED
EXECUTE TASK load_curated;                    -- run once now (testing)
```

- Run **one SQL statement, a stored procedure call, or Snowflake Scripting** on a schedule (`n MINUTE` / `SECONDS` or **CRON**) or when triggered.
- ⭐ **Created suspended**: you must `ALTER TASK … RESUME`.
- **Serverless tasks** (no `WAREHOUSE`): Snowflake sizes compute (up to XXLARGE) and you pay for actual use. Needs `EXECUTE MANAGED TASK`. Good for short, regular jobs and strict schedules.
- **User-managed tasks** (`WAREHOUSE = …`): you control size; good when a warehouse is already busy anyway.
- ⭐ Task owner needs the `EXECUTE TASK` account privilege (plus `EXECUTE MANAGED TASK` for serverless) and privileges for the SQL it runs.
- **Triggered tasks:** only `WHEN SYSTEM$STREAM_HAS_DATA(…)` and **no schedule**. They run when the stream has data.
- **Task graphs (DAGs):** a **root task** has the schedule; child tasks use `AFTER parent`. Up to **1,000 tasks** per graph. A **finalizer task** runs at the end even after failures (clean-up). Resume children before the root, or use `SYSTEM$TASK_DEPENDENTS_ENABLE('root')`.

- Only **one run at a time** by default (`ALLOW_OVERLAPPING_EXECUTION = FALSE`); a scheduled run is skipped if the previous one is still going.
- Failure handling: `SUSPEND_TASK_AFTER_NUM_FAILURES`, `TASK_AUTO_RETRY_ATTEMPTS`, `USER_TASK_TIMEOUT_MS`, `ERROR_INTEGRATION` for notifications.
- Monitor with `TASK_HISTORY()`, `COMPLETE_TASK_GRAPHS`, Snowsight Task History.

⭐ Dynamic tables

```
CREATE DYNAMIC TABLE curated.daily_sales
  TARGET_LAG = '10 minutes'          -- or DOWNSTREAM
  WAREHOUSE = transform_wh
  REFRESH_MODE = AUTO                 -- AUTO | INCREMENTAL | FULL
  INITIALIZE = ON_CREATE              -- or ON_SCHEDULE
AS
  SELECT order_date, region, SUM(amount) AS total
  FROM raw.orders JOIN raw.stores USING (store_id)
  GROUP BY order_date, region;
```

- **Declarative pipelines:** you write the **result query**; Snowflake works out how and when to refresh. No streams/tasks code to maintain.
- ⭐ `TARGET_LAG`: maximum allowed staleness vs the base tables (**minimum 60 seconds**). It's a freshness **goal**, not a schedule.
- ⭐ `TARGET_LAG = DOWNSTREAM`: refresh only when a downstream dynamic table needs it. A DOWNSTREAM table with no dynamic-table consumers **never refreshes automatically**.
- Pipeline rule: a downstream table's lag must be ≥ its upstream table's lag. Recommended: set the lag on the final table and `DOWNSTREAM` on intermediate ones.
- **Refresh modes:** `INCREMENTAL` (only changes; needs supported query constructs and change tracking), `FULL` (recompute), `AUTO` (default; incremental when possible).
- Refreshes use the **warehouse you specify** (not serverless). Refreshes run upstream first; if nothing changed upstream, the refresh is **skipped** (no cost).
- ⚠️ You **can't run DML** (INSERT/UPDATE/DELETE) against a dynamic table; its contents come from its query. You can `ALTER DYNAMIC TABLE … REFRESH`, `SUSPEND`, `RESUME`.
- Supports joins, unions, aggregates, window functions; can be chained; can be Iceberg (`CREATE DYNAMIC ICEBERG TABLE`); can be cloned.

⭐ Streams + tasks vs dynamic tables vs materialized views

	Streams + tasks	Dynamic tables	Materialized views
Style	**Imperative** (you write MERGE logic, schedules)	**Declarative** (just the query)	Declarative
Joins	Yes	**Yes**	❌ **Single table**
Freshness	Your schedule	**Target lag**	**Always current**
Compute	Warehouse or serverless task	**Your warehouse**	**Serverless**
Edition	All	All	Enterprise+
Best for	Complex procedural logic, calls to procedures, external actions, SCD logic	Multi-step SQL transformation pipelines	Speeding up repeated queries on one large table

Openflow

Snowflake's managed data-integration service (based on Apache NiFi). Not tested until globally GA.

3.3 Identify the different Snowflake connectors and integrations

⭐ Drivers and APIs

- **Drivers: JDBC**, **ODBC**, **Python** (`snowflake-connector-python`, with pandas `write_pandas`), **Node.js**, **Go**, **.NET**, **PHP PDO**. Use them from apps and BI tools.
- **Snowflake SQL API (REST):** submit SQL, check status, fetch results, cancel. Auth with OAuth, key-pair JWT or PAT.
- **Snowpark** libraries (Python, Java, Scala) and the **Snowflake Python API** (`snowflake.core`) for managing objects.
- **SnowCD**: diagnoses network connectivity to Snowflake endpoints.

⭐ Connectors

- **Snowflake Connector for Kafka:** a Kafka Connect **sink**. Writes each topic into a table with `RECORD_CONTENT` and `RECORD_METADATA` VARIANT columns (or schematised columns). Uses **Snowpipe Streaming** (recommended, lower latency and cost) or Snowpipe (creates an internal stage and pipe per partition). Key-pair authentication.
- **Snowflake Connector for Spark:** read/write between Spark and Snowflake with **query pushdown**.
- **Native App connectors** from the Marketplace: ServiceNow, Google Analytics, MySQL/PostgreSQL CDC, SharePoint and others.
- **Partner Connect:** one-click trials of ecosystem partners (Fivetran, dbt, Tableau, etc.). Creates partner objects for you (database, warehouse, role, user).

⭐ Storage integration

Lets stages reach cloud storage **without putting credentials in SQL**. Snowflake uses an identity (AWS IAM role, Azure service principal, GCS service account) that your cloud admin trusts.

```
CREATE STORAGE INTEGRATION s3_int
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = 'S3'
  ENABLED = TRUE
  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::123456789012:role/snowflake_access'
  STORAGE_ALLOWED_LOCATIONS = ('s3://my-bucket/raw/', 's3://my-bucket/exports/')
  STORAGE_BLOCKED_LOCATIONS = ('s3://my-bucket/raw/secret/');

DESC INTEGRATION s3_int;   -- copy STORAGE_AWS_IAM_USER_ARN and STORAGE_AWS_EXTERNAL_ID
                           -- into the IAM role's trust policy
GRANT USAGE ON INTEGRATION s3_int TO ROLE loader;
```

- ⭐ Integrations are **account-level objects**. Creating one needs **ACCOUNTADMIN** or the `CREATE INTEGRATION` privilege.
- Azure uses `AZURE_TENANT_ID` + a consent URL; GCS uses a Snowflake-created service account.
- ⚠️ The older alternative is `CREDENTIALS = (AWS_KEY_ID … AWS_SECRET_KEY …)` on the stage, which is discouraged.

⭐ API integration

An account-level object that stores how Snowflake calls an HTTPS API.

- **External functions:** `API_PROVIDER = aws_api_gateway | azure_api_management | google_api_gateway`, `API_AWS_ROLE_ARN`, `API_ALLOWED_PREFIXES`.
- **Git repositories:** `API_PROVIDER = git_https_api` (see below).
- Related: **external access integrations** let Python/Java/Scala UDFs and procedures reach external hosts:

```
CREATE NETWORK RULE api_rule MODE = EGRESS TYPE = HOST_PORT VALUE_LIST = ('api.example.com');
CREATE SECRET api_key TYPE = GENERIC_STRING SECRET_STRING = '…';
CREATE EXTERNAL ACCESS INTEGRATION ext_api
  ALLOWED_NETWORK_RULES = (api_rule) ALLOWED_AUTHENTICATION_SECRETS = (api_key) ENABLED = TRUE;
-- then CREATE FUNCTION … EXTERNAL_ACCESS_INTEGRATIONS = (ext_api) SECRETS = ('k' = api_key) …
```

⭐ Git integration

```
CREATE SECRET git_cred TYPE = PASSWORD USERNAME = 'me' PASSWORD = '<personal access token>';

CREATE API INTEGRATION git_api
  API_PROVIDER = git_https_api
  API_ALLOWED_PREFIXES = ('https://github.com/my-org')
  ALLOWED_AUTHENTICATION_SECRETS = (git_cred)
  ENABLED = TRUE;

CREATE GIT REPOSITORY my_repo
  API_INTEGRATION = git_api
  GIT_CREDENTIALS = git_cred
  ORIGIN = 'https://github.com/my-org/data-platform.git';

ALTER GIT REPOSITORY my_repo FETCH;                              -- pull latest
LS @my_repo/branches/main/;                                       -- browse like a stage
EXECUTE IMMEDIATE FROM @my_repo/branches/main/deploy/setup.sql;   -- run a script
```

- A **Git repository clone** in Snowflake is a schema-level object that works like a **read-only stage** with `branches/`, `tags/` and `commits/` paths.
- Supported hosts: **GitHub, GitLab, Bitbucket, Azure DevOps, AWS CodeCommit**.
- Use files as handlers for UDFs/procedures, deploy SQL scripts, and sync **Notebooks, Streamlit apps and Workspaces** (commit/push from Workspaces).

Other integration types to recognise

Integration	Purpose
Storage	External stages (S3/GCS/Azure)
API	External functions, Git
Notification	Email, webhooks, cloud queues (Snowpipe auto-ingest on Azure/GCP, error notifications)
Security	SAML2 SSO, Snowflake OAuth, External OAuth, SCIM
External access	Outbound network calls from UDFs/procedures
Catalog	Iceberg tables with external catalogs
External volume (account object)	Storage location for Iceberg tables

✅ Domain 3: must remember

- Load: CSV, JSON, Avro, ORC, Parquet, XML. **Unload: CSV, JSON, Parquet only.**
- Stages: `@~` user, `@%t` table (no transformations), `@name` named. **PUT/GET internal only**, from a client, PUT gzips by default.
- `SNOWFLAKE_SSE` needed for file URLs on internal stages; `SNOWFLAKE_FULL` is default; can't change after creation.
- COPY load metadata **64 days**; Snowpipe **14 days**. `FORCE` reloads; `TRUNCATE` resets load metadata.
- ON_ERROR default: **ABORT_STATEMENT (COPY)**, **SKIP_FILE (Snowpipe)**. `VALIDATION_MODE` = dry run. `PURGE` deletes loaded files.
- COPY transformations: no WHERE / JOIN / GROUP BY / FLATTEN.
- Files **100–250 MB compressed**. Unload default 16 MB per file, `SINGLE = TRUE` up to 5 GB.
- Snowpipe: serverless, event notifications (SQS / Event Grid / Pub/Sub) or REST, **credits per GB**. Snowpipe Streaming: **rows**, seconds, SDK/Kafka.
- Streams: offset advances **only on committed DML**; standard / append-only / insert-only (external tables); staleness extended up to 14 days.
- Tasks: **created suspended**, `EXECUTE TASK` privilege, serverless or warehouse, DAG root + `AFTER`, finalizer.
- Dynamic tables: **TARGET_LAG** (min 60 s), `DOWNSTREAM`, refresh on your warehouse, **no DML**, joins allowed.
- Storage integration avoids credentials; needs ACCOUNTADMIN / `CREATE INTEGRATION`; `DESC INTEGRATION` gives the IAM user ARN + external ID.
- Git: API integration (`git_https_api`) + secret + `CREATE GIT REPOSITORY`; `FETCH`; `EXECUTE IMMEDIATE FROM`.

Domain 4 — Performance Optimization, Querying, and Transformation (21%)

4.1 Evaluate query performance

⭐ Query Profile (Snowsight → Query History → a query → Query Profile)

A graph of **operator nodes** (bottom-up) with statistics.

- **Common operators:** `TableScan`, `Filter`, `JoinFilter`, `Join`, `Aggregate`, `GroupingSets`, `WindowFunction`, `Sort`, `SortWithLimit`, `Flatten`, `UnionAll`, `ExternalScan` (stage/external table), `Insert` / `Update` / `Delete` / `Merge`, `Result`.
- **Profile overview:** where time went: Processing, **Local Disk IO**, **Remote Disk IO**, **Network Communication**, Synchronization, Initialization.
- **Statistics:** Bytes scanned, **Percentage scanned from cache**, **Partitions scanned vs Partitions total**, **Bytes spilled to local storage**, **Bytes spilled to remote storage**, bytes sent over network, rows produced per node.

⭐ Query insights

Snowflake flags problems automatically, both in the **Query Profile ("Query Insights" pane)** and in `ACCOUNT_USAGE.QUERY_INSIGHTS`. Topics:

- **Table scan / pruning:** no filter on a large table scan, filter not selective, filter not applicable, `LIKE '%…'` with a leading wildcard. Positive insights show when a clustering key or search optimization helped.
- **Joins: exploding join**, join with no join condition (cartesian), inefficient join condition.
- **Aggregation / union:** unnecessary `DISTINCT` / `GROUP BY`, `UNION` where `UNION ALL` would do.
- **Warehouse: remote spillage**, **query queued too long**.

⭐ The four classic problems

Symptom in the profile	What it means	Fix
Bytes spilled to local storage	Not enough memory; intermediate results written to the warehouse's local SSD	**Larger warehouse (scale up)**, process less data (filter early, select fewer columns), split the work into batches
Bytes spilled to remote storage	Worse: local disk also full; written to cloud storage. Very slow.	Same fixes, more urgently. **Scale up.**
Inefficient pruning (partitions scanned ≈ partitions total)	Filters don't match how data is clustered	Filter on clustered columns, add a **clustering key**, avoid wrapping filter columns in functions, **search optimization** for point lookups
Exploding join (join outputs far more rows than inputs)	Missing/wrong join condition, many-to-many keys	Fix the join predicate, de-duplicate or aggregate before joining
Queuing (`QUEUED_OVERLOAD_TIME` high)	Warehouse is **overloaded** by concurrent queries	**Multi-cluster / scale out**, separate workloads onto different warehouses
`QUEUED_PROVISIONING_TIME` high	Waiting for the warehouse to start or resize	Keep it running for latency-sensitive work, or accept
Slow `ORDER BY` without `LIMIT`, huge `SORT`	Large sort spilling	Add `LIMIT` / `QUALIFY`, sort less data

⭐ SNOWFLAKE.ACCOUNT_USAGE views for performance

`QUERY_HISTORY` (365 days, ~45-minute latency). Columns to know: `QUERY_ID`, `QUERY_TEXT`, `QUERY_TAG`, `USER_NAME`, `ROLE_NAME`, `WAREHOUSE_NAME`, `WAREHOUSE_SIZE`, `EXECUTION_STATUS`, `TOTAL_ELAPSED_TIME`, `COMPILATION_TIME`, `EXECUTION_TIME`, `QUEUED_OVERLOAD_TIME`, `QUEUED_PROVISIONING_TIME`, `BYTES_SCANNED`, `PERCENTAGE_SCANNED_FROM_CACHE`, `PARTITIONS_SCANNED`, `PARTITIONS_TOTAL`, `BYTES_SPILLED_TO_LOCAL_STORAGE`, `BYTES_SPILLED_TO_REMOTE_STORAGE`, `ROWS_PRODUCED`, `CREDITS_USED_CLOUD_SERVICES`, `QUERY_HASH`, `QUERY_PARAMETERIZED_HASH`.

- Real-time alternative for the last 7 days: `INFORMATION_SCHEMA.QUERY_HISTORY()`, `QUERY_HISTORY_BY_USER()`, `QUERY_HISTORY_BY_WAREHOUSE()`, `QUERY_HISTORY_BY_SESSION()`.
- `QUERY_HASH` / `QUERY_PARAMETERIZED_HASH` group repeated queries (the parameterized one ignores literal values).

```
-- Top 10 queries that spilled to remote storage this week
SELECT query_id, warehouse_name, bytes_spilled_to_remote_storage, total_elapsed_time/1000 AS secs
FROM snowflake.account_usage.query_history
WHERE start_time > DATEADD(day, -7, CURRENT_TIMESTAMP())
  AND bytes_spilled_to_remote_storage > 0
ORDER BY bytes_spilled_to_remote_storage DESC
LIMIT 10;
```

⭐ Query attribution: `QUERY_ATTRIBUTION_HISTORY`

- **Estimated warehouse compute credits per query** (`CREDITS_ATTRIBUTED_COMPUTE`, plus `CREDITS_USED_QUERY_ACCELERATION`), with `USER_NAME`, `QUERY_TAG`, `WAREHOUSE_NAME`, `ROOT_QUERY_ID` / `PARENT_QUERY_ID` (for procedures).
- **Excludes** warehouse **idle time**, cloud services, storage, data transfer, serverless and AI costs, and very short queries (~100 ms or less).
- Latency up to about 8 hours; 365 days retained.
- Use it to **charge back cost by user, team (`QUERY_TAG`) or workload**.

```
SELECT user_name, SUM(credits_attributed_compute) AS credits
FROM snowflake.account_usage.query_attribution_history
WHERE start_time >= DATE_TRUNC('month', CURRENT_DATE())
GROUP BY user_name ORDER BY credits DESC;
```

- **Query tagging:** `ALTER SESSION SET QUERY_TAG = 'team=finance;job=daily_close';` makes queries easy to group in history and attribution views. Object **tags on warehouses** attribute warehouse cost.

⭐ Workload management best practices

- **Group similar workloads** on the same warehouse (similar size, complexity and SLA). Mixing tiny dashboard queries with huge ETL on one warehouse causes queuing and wasted capacity.
- **Separate warehouses** for ETL/loading, BI, data science and ad-hoc; size each for its workload.
- BI with many users → **multi-cluster**. Heavy single jobs → **bigger size**. Occasional outlier scans → **QAS**.
- Set **`STATEMENT_TIMEOUT_IN_SECONDS`** to kill runaway queries and **`STATEMENT_QUEUED_TIMEOUT_IN_SECONDS`** to cap waiting.
- Use **resource monitors** and **budgets** for cost guardrails.

4.2 Optimize query performance

⭐ Side-by-side comparison

Feature	Edition	Best for	How it works	Cost
Clustering key + Automatic Clustering	All	**Very large tables (multi-TB)** filtered or joined on the **same few columns**, often **range** filters (dates)	Reorganises micro-partitions so values are co-located → better pruning	**Serverless** reclustering credits + extra storage from rewritten partitions
Search Optimization Service (SOS)	**Enterprise**	**Selective point lookups** returning few rows: `=`, `IN`, **substring/regex** (`LIKE`, `ILIKE`, `RLIKE`), **VARIANT** fields, **geospatial**, `SEARCH()` full-text	Builds a persistent **search access path** (index-like structure)	Serverless build/maintenance + **storage** for the access path
Materialized views	**Enterprise**	Same **expensive aggregation/projection on one table** repeated often; data changes rarely; speeding up **external tables**	Pre-computed, stored results; optimizer can **rewrite** queries to use them automatically	Serverless refresh + storage
Query Acceleration Service (QAS)	**Enterprise**	**Outlier / ad-hoc queries** that **scan lots of data with selective filters or aggregations**; unpredictable workloads	Offloads parts of the query (scans, filters) to shared **serverless** compute	Serverless credits, capped by **scale factor**

⭐ Clustering keys

```
ALTER TABLE events CLUSTER BY (event_date, TO_DATE(event_ts), customer_region);
SELECT SYSTEM$CLUSTERING_INFORMATION('events', '(event_date)');
ALTER TABLE events SUSPEND RECLUSTER;   ALTER TABLE events RESUME RECLUSTER;
ALTER TABLE events DROP CLUSTERING KEY;
```

- **When:** table is **large (multi-terabyte)**, queries **filter/join on the same columns**, query performance or pruning has **degraded**, and the table is **queried far more often than it's changed**.
- **Choosing columns:** at most **3–4 columns**; prefer columns used in **selective filters**, then join columns; order **low cardinality → high cardinality**. Avoid extremely high-cardinality columns (raw timestamps, IDs) — use an **expression** like `TO_DATE(ts)` — and avoid very low cardinality (booleans).
- **Automatic Clustering** is serverless and runs in the background once a key is defined; you can suspend/resume it.
- ⚠️ Frequent DML on a clustered table causes **constant reclustering cost**.

⭐ Search Optimization Service

```
ALTER TABLE customers ADD SEARCH OPTIMIZATION;                          -- whole table
ALTER TABLE logs ADD SEARCH OPTIMIZATION ON EQUALITY(user_id), SUBSTRING(message),
EQUALITY(payload:device.id);
SELECT SYSTEM$ESTIMATE_SEARCH_OPTIMIZATION_COSTS('customers');
```

- For "**needle in a haystack**" queries that return a **small number of rows** from a big table, especially on **high-cardinality columns** where clustering can't help.
- Needs `OWNERSHIP` on the table and `ADD SEARCH OPTIMIZATION` on the schema.
- ⚠️ Doesn't help queries that return most of the table or do large range scans (use clustering for those).

⭐ Materialized views

```
CREATE MATERIALIZED VIEW mv_daily_revenue
  CLUSTER BY (sale_date) AS
SELECT sale_date, store_id, SUM(amount) AS revenue, COUNT(*) AS n
FROM sales
GROUP BY sale_date, store_id;
```

- **Always current:** if background refresh is behind, Snowflake combines the MV with the newest base-table changes at query time.
- ⚠️ **Limitations: one table only (no joins, no self-joins)**; no window functions, `HAVING`, `ORDER BY`, `LIMIT`, `UNION` / `MINUS` / `INTERSECT`, nested subqueries, UDFs, or non-deterministic functions; only certain aggregates (e.g., `SUM`, `COUNT`, `MIN`, `MAX`, `AVG`, `APPROX_COUNT_DISTINCT`).
- Use a **dynamic table** instead when you need joins or a defined lag.
- An MV can have its own **clustering key**. Secure MVs hide the definition.
- Monitor refresh cost: `MATERIALIZED_VIEW_REFRESH_HISTORY`.

⭐ Query Acceleration Service

```
ALTER WAREHOUSE adhoc_wh SET ENABLE_QUERY_ACCELERATION = TRUE
                             QUERY_ACCELERATION_MAX_SCALE_FACTOR = 8;   -- default 8; 0 = no limit
SELECT SYSTEM$ESTIMATE_QUERY_ACCELERATION('<query_id>');                -- is this query eligible?
SELECT * FROM snowflake.account_usage.query_acceleration_eligible;      -- candidates
```

- **Scale factor** = maximum serverless compute QAS may lease, as a **multiple of the warehouse size**.
- Helps **eligible** queries: large scans with selective filters or aggregations, `INSERT … SELECT`, CTAS, `COPY INTO`. Doesn't help queries with little scanning, non-selective filters or non-deterministic filter functions.
- ⭐ **QAS fixes outliers on an otherwise right-sized warehouse**; it's not a replacement for scaling up (spilling) or out (concurrency).
- Enabled by default (scale factor 2) on **new Gen2** and **new multi-cluster** warehouses.
- Monitor: `QUERY_ACCELERATION_HISTORY`, `QUERY_ATTRIBUTION_HISTORY.CREDITS_USED_QUERY_ACCELERATION`.

*Also newer: **Snowflake Optima** applies automatic optimisations (such as automatic indexing) on Gen2 warehouses; you'll see them as positive query insights.*

4.3 Use Snowflake caching

⭐ The three caches

Cache	Lives in	Holds	Lifetime	Needs a warehouse?
Query result cache	**Cloud services**	Final **result sets** of queries	**24 hours**, reset each time it's reused, up to **31 days** max	**No**
Metadata cache	**Cloud services**	Micro-partition statistics: **row counts, MIN/MAX, distinct counts, NULL counts**; object definitions	Always maintained	**No** (`COUNT(*)`, `MIN`, `MAX`, `SHOW`, `DESCRIBE`)
Warehouse cache (local disk / SSD)	**Each running warehouse**	Raw **table data** (column files) read from storage	Until the warehouse **suspends** (lost) or resizes (reduced)	Yes (it's part of the warehouse)

⭐ Result cache reuse rules

A new query reuses a cached result only when **all** of these hold:

- The **query text is identical** (⚠️ case, spacing and aliases matter: `select` vs `SELECT` = different).
- **Underlying data hasn't changed** (and micro-partitions weren't reclustered).
- It doesn't use **runtime/non-deterministic functions** (`RANDOM()` , `UUID_STRING()` , `CURRENT_TIMESTAMP()`), **external functions** or **hybrid tables**.
- The role has **privileges on all the tables** (results can be shared across users with the same access).
- Relevant configuration/session settings haven't changed, and the result is still retained.

Controls: **`USE_CACHED_RESULT`** (default TRUE; set FALSE at account/user/session level to force re-execution when benchmarking).

`RESULT_SCAN` reads a previous result as a table, handy after `SHOW` / `DESCRIBE` / `CALL` :

```
SHOW WAREHOUSES;
SELECT "name", "size" FROM TABLE(RESULT_SCAN(LAST_QUERY_ID())) WHERE "state" = 'STARTED';
```

Warehouse cache trade-off: a short `AUTO_SUSPEND` saves credits but throws away the SSD cache; for BI dashboards that rerun similar queries, a longer auto-suspend can be faster and not much more expensive. Check `PERCENTAGE_SCANNED_FROM_CACHE` .

4.4 Perform data transformation techniques

⭐ Structured data

Standard SQL types: `NUMBER(p,s)` (= `DECIMAL` / `INT`), `FLOAT` , `VARCHAR` (max 128 MB now), `BOOLEAN` , `DATE` , `TIME` , `TIMESTAMP_NTZ` / `TIMESTAMP_LTZ` / `TIMESTAMP_TZ` , `BINARY` , `GEOGRAPHY` / `GEOMETRY` , `VECTOR` .

- Constraints: **only `NOT NULL` is enforced** on standard tables. PRIMARY KEY, UNIQUE and FOREIGN KEY are informational (hybrid tables enforce PKs).
- Useful DML: `MERGE` (upserts), **multi-table `INSERT ALL / INSERT FIRST`** , `INSERT OVERWRITE` , CTAS, `CREATE TABLE … LIKE` (structure only), `CREATE TABLE … CLONE` .
- ⭐ `TRUNCATE TABLE` removes all rows as a fast metadata operation (and clears load history). `DELETE` without `WHERE` is slower and keeps load history. `DROP` removes the table (recoverable with `UNDROP` in Time Travel).
- Transactions: `BEGIN … COMMIT/ROLLBACK` ; **DDL auto-commits**; `AUTOCOMMIT` default TRUE.

⭐ Semi-structured data (JSON, Avro, Parquet, ORC, XML)

Types: **`VARIANT`** (any value, up to **128 MB** uncompressed; older material says 16 MB), **`OBJECT`** (key–value), **`ARRAY`** .
Structured versions: `ARRAY(INT)` , `OBJECT(name STRING, age INT)` , `MAP(VARCHAR, INT)` .

```
-- raw = {"id": 7, "customer": {"name": "Ana", "tier": "gold"},
--        "items": [{"sku": "A1", "qty": 2}, {"sku": "B7", "qty": 1}]}
SELECT
  raw:id::INT                    AS order_id,
  raw:customer.name::STRING      AS customer_name,   -- dot for nested levels
  raw['customer']['tier']::STRING AS tier,           -- bracket notation
  i.value:sku::STRING            AS sku,
  i.value:qty::INT               AS qty,
  i.index                        AS line_no
FROM orders_raw,
     LATERAL FLATTEN(INPUT => raw:items) i;
```

- `:` gets a first-level field, `.` goes deeper, `[ ]` for array positions or awkward names, `::` **casts** (and removes the JSON double quotes from strings).
- ⚠️ **Field names in the path are case-sensitive**; the column name (`raw`) isn't.
- ⭐ **FLATTEN** turns an array/object into rows. Output columns: `SEQ`, `KEY`, `PATH`, `INDEX`, `VALUE`, `THIS`. Arguments: `INPUT`, `PATH`, `OUTER => TRUE` (keep rows whose array is empty or missing), `RECURSIVE => TRUE` (expand all nested levels), `MODE` (`'OBJECT'`, `'ARRAY'`, `'BOTH'`).
- **JSON null vs SQL NULL:** a JSON `null` becomes a **VARIANT null** (displayed `null`), which **isn't** SQL NULL. `STRIP_NULL_VALUES = TRUE` drops them on load; `IS_NULL_VALUE()` tests them.
- **Functions:** `PARSE_JSON`, `TRY_PARSE_JSON`, `TO_JSON`, `OBJECT_CONSTRUCT` (skips NULLs; `OBJECT_CONSTRUCT_KEEP_NULL` keeps them), `ARRAY_CONSTRUCT`, `ARRAY_AGG`, `OBJECT_AGG`, `ARRAY_SIZE`, `ARRAY_CONTAINS`, `ARRAY_FLATTEN`, `GET`, `GET_PATH`, `OBJECT_KEYS`, `OBJECT_INSERT`, `TYPEOF`, `IS_OBJECT` / `IS_ARRAY`, `CHECK_JSON`, `PARSE_XML`, `XMLGET`, `STRIP_NULL_VALUE`.
- **Performance:** Snowflake automatically stores frequently used, consistently typed paths in a columnar way so they can be pruned. **Extract frequently queried fields into typed columns** for the best performance, especially **dates/timestamps** (stored as strings in JSON).
- Loading semi-structured: into one VARIANT column, or into columns with `MATCH_BY_COLUMN_NAME`, or with `INFER_SCHEMA` + `USING TEMPLATE`.

⭐ Unstructured data (PDFs, images, audio, video)

- Store files in an internal or external **stage**; add a **directory table** to catalogue them.
- ⭐ **Three URL types:**

URL	Function	Expires	Who can use it
Scoped URL	`BUILD_SCOPED_FILE_URL(@stage, 'path')`	**24 hours** (result cache period)	**Only the user who generated it**; good for apps and Streamlit
File URL	`BUILD_STAGE_FILE_URL(@stage, 'path')`	**Permanent**	Roles with stage privileges (`USAGE` external / `READ` internal)
Pre-signed URL	`GET_PRESIGNED_URL(@stage, 'path', expiration_seconds)`	You choose	**Anyone with the link**, no Snowflake login (e.g., BI tools showing images)

- ⚠️ For internal stages, these URLs require **server-side encryption (`SNOWFLAKE_SSE`)**.
- `FILE` **data type** + `TO_FILE('@stage/path')` represents a staged file in tables and AI functions.
- **Processing:** Java/Python **UDFs, UDTFs and stored procedures** read files (`SnowflakeFile` class); **Cortex** functions (`AI_PARSE_DOCUMENT`, `AI_EXTRACT`, `AI_TRANSCRIBE`, `AI_COMPLETE` on images); **Document AI**; external functions.

⭐ Aggregate functions

- Basics: `COUNT(*)` vs `COUNT(col)` (ignores NULLs), `COUNT(DISTINCT col)`, `COUNT_IF(cond)`, `SUM`, `AVG`, `MIN`, `MAX`, `MEDIAN`, `MODE`, `STDDEV`, `LISTAGG(col, ',')`, `ARRAY_AGG`, `OBJECT_AGG`, `ANY_VALUE`.
- ⭐ **Approximate (fast on huge data):** `APPROX_COUNT_DISTINCT` / `HLL` (**HyperLogLog**), `APPROX_TOP_K`, `APPROX_PERCENTILE`, `MINHASH` / `APPROXIMATE_SIMILARITY`.
- Grouping: `GROUP BY`, **`GROUP BY ALL`** (groups by all non-aggregate select items), `GROUP BY ROLLUP / CUBE / GROUPING SETS`, `HAVING` (filters groups after aggregation).

⭐ Window functions

```
SELECT customer_id, order_date, amount,
       ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS rn,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
       LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_date)     AS prev_amount,
       RANK() OVER (ORDER BY amount DESC) AS amount_rank
FROM orders
QUALIFY rn = 1;      -- latest order per customer, without a subquery
```

- A window function computes over a set of related rows **without collapsing them** (unlike GROUP BY).
- **Ranking:** `ROW_NUMBER` (unique 1,2,3), `RANK` (gaps after ties: 1,1,3), `DENSE_RANK` (no gaps: 1,1,2), `NTILE(n)`, `PERCENT_RANK`, `CUME_DIST`.
- **Offset:** `LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`, `NTH_VALUE`.
- **Aggregate as window:** `SUM/AVG/COUNT/MIN/MAX … OVER (…)` with frames `ROWS` or `RANGE BETWEEN …`. With `ORDER BY` and no frame, the default is a **cumulative** frame (unbounded preceding to current row).
- ⭐ **`QUALIFY`** filters on window function results (like `HAVING` for windows). Classic use: **de-duplicate** / keep the latest row per key.

⭐ Applying SQL for query optimization

- **Select only needed columns** (columnar storage: `SELECT *` reads everything). `SELECT * EXCLUDE (col)` and `RENAME` help.
- **Filter early and on clustered/naturally ordered columns** (e.g., date) to maximise **pruning**. Avoid functions on the filter column (`WHERE TO_DATE(ts) = …` prunes worse than `WHERE ts >= … AND ts < …`).
- Avoid **leading wildcards** (`LIKE '%abc'`) unless search optimization covers them.
- Use **`UNION ALL`** instead of `UNION` when duplicates are impossible or acceptable.
- Join on proper keys; **aggregate or de-duplicate before joining** to avoid exploding joins; avoid cartesian joins.
- Use `QUALIFY` instead of a self-join or subquery for "top N per group".
- Use `LIMIT` with `ORDER BY` for top-N; avoid sorting huge results needlessly.
- Use **`TRUNCATE`** rather than `DELETE` to empty a table; **CTAS** or `INSERT OVERWRITE` to rebuild.
- Use **approximate functions** for large distinct counts and percentiles.
- **Sampling:** `SAMPLE (10)` / `TABLESAMPLE BERNOULLI (10)` (row-level) vs `SYSTEM (10)` / `BLOCK` (micro-partition-level, faster, less random).
- `EXPLAIN` shows the plan without running the query.
- Re-use results: result cache, materialized views, dynamic tables, temporary/transient tables for intermediate steps.

✅ Domain 4: must remember

- Query Profile: **spilling → scale up**; **partitions scanned ≈ total → pruning/clustering**; **join rows explode → fix join**; **queued overload → scale out**.
- **Query insights** = automatic tips (Query Profile pane and `QUERY_INSIGHTS` view).
- `QUERY_ATTRIBUTION_HISTORY` = **credits per query** (no idle time). `QUERY_TAG` for chargeback.
- **Clustering** = big tables, range filters. **SOS** = point lookups, substring, VARIANT, geo. **MV** = repeated aggregations on one table. **QAS** = outlier scans. SOS/MV/QAS need **Enterprise**.
- QAS scale factor default **8** (2 on new Gen2/multi-cluster warehouses); `SYSTEM$ESTIMATE_QUERY_ACCELERATION`.
- Result cache: **24 h, up to 31 days**, cloud services, **no warehouse**, identical text, unchanged data, no `RANDOM` / `CURRENT_TIMESTAMP`. `USE_CACHED_RESULT = FALSE` to disable.
- Metadata cache answers `COUNT(*)`, `MIN`, `MAX` without a warehouse. Warehouse cache is **lost on suspend**.
- VARIANT up to 128 MB; `:` `.` `[]` `::`; FLATTEN columns **SEQ, KEY, PATH, INDEX, VALUE, THIS**; `OUTER => TRUE` keeps empty arrays.
- Scoped URL **24 h, generating user only**; file URL **permanent, needs privileges**; pre-signed URL **anyone, custom expiry**.
- `QUALIFY` filters window results; `RANK` has gaps, `DENSE_RANK` doesn't; `APPROX_COUNT_DISTINCT` = HyperLogLog.

Domain 5 — Data Collaboration (10%)

5.1 Explain data collaboration and protection

⭐ Continuous data protection: the lifecycle

```
 Current data  ──DML/DROP──►  TIME TRAVEL (you can query, clone, UNDROP)  ──►  FAIL-SAFE (Snowflake Support only)  ──►  gone
                                  0–1 day Standard / 0–90 days Enterprise+         7 days, permanent tables only
```

⭐ Time Travel

- Access data **as it was** at an earlier point, **restore dropped objects**, and **clone from the past**.
- `DATA_RETENTION_TIME_IN_DAYS`: default **1**. Standard: 0–1. **Enterprise+: 0–90 for permanent objects.** Transient and temporary: 0–1. `0` disables Time Travel.
- Set at account, database, schema or table level (most specific wins). `MIN_DATA_RETENTION_TIME_IN_DAYS` (account, ACCOUNTADMIN) sets a floor: effective retention = the **larger** of the two.

```
SELECT * FROM orders AT (TIMESTAMP => '2026-10-20 09:00:00'::TIMESTAMP_LTZ);
SELECT * FROM orders AT (OFFSET => -60*30);                    -- 30 minutes ago
SELECT * FROM orders BEFORE (STATEMENT => '01b2c3d4-0000-…');   -- just before that query ran

CREATE TABLE orders_restored CLONE orders AT (OFFSET => -3600);

DROP TABLE orders;
UNDROP TABLE orders;          -- also UNDROP SCHEMA / UNDROP DATABASE (and more object types)
SHOW TABLES HISTORY LIKE 'orders%';   -- dropped versions appear with DROPPED_ON
```

- ⚠️ `UNDROP` fails if an object with the same name exists: **rename the current one first**, then UNDROP.
- ⚠️ When a **database or schema is dropped**, child objects are kept for the **container's** retention period, even if a child had a longer setting.
- Reducing retention moves older data into Fail-safe; increasing it applies to data still in Time Travel.
- Time Travel data is billed as storage. Streams can extend retention (see 3.2).

⭐ Fail-safe

- A **non-configurable 7-day** period **after Time Travel ends**, for **permanent** tables only (transient/temporary: **none**).
- ⭐ **Not accessible to users**. Data can be recovered **only by Snowflake Support**, on a best-effort basis (hours to days). It's for disasters, not routine restores.
- Costs storage. Check with `TABLE_STORAGE_METRICS` (`ACTIVE_BYTES`, `TIME_TRAVEL_BYTES`, `FAILSAFE_BYTES`, `RETAINED_FOR_CLONE_BYTES`).
- To avoid Fail-safe cost for rebuildable data, use **transient** tables.

⭐ Zero-copy cloning

```
CREATE DATABASE dev_db CLONE prod_db;
CREATE SCHEMA test.sales CLONE prod.sales AT (TIMESTAMP => '2026-10-23 00:00'::TIMESTAMP_LTZ);
CREATE TABLE orders_copy CLONE orders COPY GRANTS;
CREATE DATABASE dev2 CLONE prod_db INCLUDE INTERNAL STAGES;
```

- A clone is a **metadata-only copy**: it points to the same micro-partitions and **uses no extra storage** until either side changes data. Clones are **independent** afterwards.
- ⭐ **Cloneable:** databases, schemas, tables (incl. dynamic and Snowflake-managed Iceberg), streams, tasks, **named external stages**, file formats, sequences, pipes, alerts, database roles.
- ⭐ **Rules that get tested:**
  - **Named internal stages** are cloned only with `INCLUDE INTERNAL STAGES`. **Table stages** are cloned **empty**.
  - **Pipes** that reference an **internal stage are skipped**; external-stage pipes are cloned (paused, or `STOPPED_CLONED` for auto-ingest pipes).
  - **Tasks** and **alerts** in a clone are **suspended**; resume them yourself.
  - **Streams:** unconsumed records are **not** available in the clone (official sample Q2).
  - **Privileges:** cloning a database/schema copies grants on the **child objects** but **not on the container itself**. A single-table clone copies **no grants** unless you use `COPY GRANTS`.
  - **Automatic Clustering is suspended** on a cloned table until you `RESUME RECLUSTER`.
  - Masking/row access **policies can't be cloned individually**; cloning the schema clones them.
  - Cloning with Time Travel (`AT` / `BEFORE`) fails if a child's retention doesn't reach that point (use `IGNORE TABLES WITH INSUFFICIENT DATA RETENTION`).
- Use cases: instant **dev/test environments**, backups before risky changes, point-in-time recovery.

⭐ Data replication and failover

Concept	Meaning
Primary / secondary	Objects are replicated from a **primary** (read-write) to **secondaries** (**read-only**) in other accounts of the **same organization**, in any region or cloud
Database replication	Replicate databases (and shares). **All editions.**
Replication group	A set of objects replicated together with **point-in-time consistency** (databases, shares, and in Business Critical also users, roles, warehouses, integrations, network policies, resource monitors…). Scheduled with `REPLICATION_SCHEDULE`.
Failover group	A replication group that can **fail over**: promote a secondary to primary (`ALTER FAILOVER GROUP fg PRIMARY;`). **Business Critical+.**
Client redirect	**Connection objects** give clients one URL (`org-connection.snowflakecomputing.com`) that you repoint to the new primary during an outage. **Business Critical+.**

```
-- in the source account
CREATE FAILOVER GROUP fg
  OBJECT_TYPES = DATABASES, ROLES, USERS, WAREHOUSES
  ALLOWED_DATABASES = sales_db
  ALLOWED_ACCOUNTS = myorg.dr_account
  REPLICATION_SCHEDULE = '10 MINUTE';
-- in the target account
CREATE FAILOVER GROUP fg AS REPLICA OF myorg.prod_account.fg;
ALTER FAILOVER GROUP fg REFRESH;     -- manual refresh
ALTER FAILOVER GROUP fg PRIMARY;     -- fail over
```

- Replication must be **enabled for the accounts by an organization administrator** (ORGADMIN / GLOBALORGADMIN).
- **Costs: data transfer** (cross-region/cloud egress) + **serverless compute** for refreshes + **storage** in the target. Monitor with `REPLICATION_GROUP_USAGE_HISTORY`.
- ⚠️ Not replicated: **databases created from shares** (inbound shares), temporary tables, external tables, hybrid tables. Secondary objects are read-only until promoted.
- Replication is also how you **share data across regions** without listings (replicate, then share locally).

Secure data sharing features (overview)

Covered in 5.2: shares, reader accounts, listings, Marketplace, clean rooms, Native Apps. All of them build on the same idea: **share live data without copying it**.

5.2 Explain Snowflake's data sharing capabilities

⭐ How Secure Data Sharing works

- **No data is copied or moved.** The consumer reads the provider's micro-partitions through the **services layer**. Shared data is **live** (changes appear instantly) and access can be **revoked any time**.
- ⭐ **Provider** pays for **storage**. **Consumer** pays for the **compute** (their warehouses) used to query it. Consumers pay **no storage** for shared data.
- Shared objects are **read-only** for consumers.

⭐ Account roles in sharing

Account	Role
Provider	Creates a **share**, grants objects to it, adds consumer accounts
Consumer	Creates a **read-only database from the share** and grants access to its roles. Any full account can be both provider and consumer.
Reader account	⭐ A **managed account created by the provider** for a consumer that **has no Snowflake account**. The **provider pays** all its compute. It can **only query** data shared by **that provider**: **no loading, no DML**, no own data. Created with `CREATE MANAGED ACCOUNT … TYPE = READER`. Monitor with `READER_ACCOUNT_USAGE`.

⭐ Direct shares, step by step

```
-- PROVIDER (ACCOUNTADMIN or a role with CREATE SHARE)
CREATE SHARE sales_share;
GRANT USAGE  ON DATABASE sales_db                        TO SHARE sales_share;
GRANT USAGE  ON SCHEMA   sales_db.public                 TO SHARE sales_share;
GRANT SELECT ON TABLE    sales_db.public.orders          TO SHARE sales_share;
GRANT SELECT ON VIEW     sales_db.public.v_orders_secure TO SHARE sales_share;
GRANT REFERENCE_USAGE ON DATABASE ref_db TO SHARE sales_share;  -- view uses objects in another DB
ALTER SHARE sales_share ADD ACCOUNTS = partner_org.partner_acct;

-- CONSUMER (ACCOUNTADMIN or a role with IMPORT SHARE)
SHOW SHARES;
CREATE DATABASE partner_sales FROM SHARE provider_org.provider_acct.sales_share;
GRANT IMPORTED PRIVILEGES ON DATABASE partner_sales TO ROLE analyst;
```

- **What can be shared:** databases, tables, dynamic tables, external tables, Iceberg tables, **secure views**, secure materialized views, **secure UDFs**, semantic views, Cortex Search services and some model types.
- ⭐ Share views as **secure views** so consumers can't see the definition or underlying data (non-secure views need the share to allow them).
- ⭐ A share can include objects from **one database** (plus others through `REFERENCE_USAGE` for views). A consumer creates **one database per share**.
- **Database roles in shares:** grant privileges to a database role, then `GRANT DATABASE ROLE … TO SHARE`. The consumer grants that database role to its own roles, so different consumer users get different subsets (finer than `IMPORTED PRIVILEGES`, which is all-or-nothing).
- **Row-level control per consumer:** use `CURRENT_ACCOUNT()` in a secure view or row access policy with a mapping table.
- ⚠️ **Consumer limits:** read-only; **can't clone** a shared database; **no Time Travel** on shared objects; **can't re-share** (see below).
- ⭐ **Direct shares work only within the same region.** For other regions or clouds, use a **listing with Cross-Cloud Auto-Fulfillment**, or **replicate** the database to an account in that region and share from there.
- Sharing from a Business Critical account to a lower edition is blocked by default (needs `OVERRIDE SHARE RESTRICTIONS`).

⭐ Sharing and resharing

- **Direct shares:** consumers **can't re-share** imported data with other accounts (a newer behaviour change lets consumers reshare to accounts **inside their own organization**; cross-organization resharing still needs a listing).
- **Listings:** the provider controls resharing in the **listing manifest**.
- If a consumer needs to pass data on, the usual answer is: **the provider shares directly with the third party**, or uses a listing.

Direct shares vs listings

	Direct share	Listing
Regions	**Same region only**	**Any region/cloud** (Cross-Cloud Auto-Fulfillment)
Audience	Specific accounts	Specific accounts/orgs (**private**) or everyone (**Marketplace**)
Metadata	None	Title, description, sample queries, docs
Monetisation	No	**Free, limited trial, paid**

	Direct share	Listing
Usage analytics	No	Yes (consumer usage metrics)
Can contain	Data	Data **or a Native App**

⭐ Data clean rooms

- A **privacy-safe collaboration** environment: two or more parties **analyse their combined data without exposing raw rows** to each other (e.g., advertiser + publisher audience overlap).
- **Snowflake Data Clean Rooms** (managed product in Snowsight, plus a Collaboration API):
  - **Owner** creates the collaboration and invites collaborators.
  - **Data providers** offer tables/views (**data offerings**) and choose who may use them.
  - **Analysis runners** run **approved templates** (parameterised **Jinja SQL**) on those offerings and **pay for the compute**.
- Protection comes from **templates**, plus **aggregation, projection and join policies**, **differential privacy** and secure sharing underneath.
- Data providers need **Enterprise Edition** (for policy enforcement); runners can be on Standard. Cross-region collaborators use Cross-Cloud Auto-Fulfillment.

5.3 Share data using the Snowflake Marketplace and listings

⭐ Listings

- A listing **wraps a share or a Native App** with metadata and access rules. It's the "enhanced" way to share.
- **Private listing:** visible only to **accounts/organizations you choose**, in any region.
- **Public listing:** published on the **Snowflake Marketplace** for everyone (after Snowflake review).
- **Access types: Free**, **Limited trial** (Marketplace only, 1–90 days, then request full access), **Paid** (pricing plans and offers; supported regions only).
- **Organizational listings** share data products **inside your organization**.
- **Cross-Cloud Auto-Fulfillment:** Snowflake automatically replicates the listing's data to consumers' regions (provider pays the replication cost).
- Providers manage listings in **Provider Studio**; consumers find them in **Snowsight → Marketplace** and click **Get** (creates a database or installs an app).

⭐ Snowflake Marketplace

- A catalogue of **live, ready-to-query data, Native Apps and AI products** from third parties (weather, financial, demographic data, connectors…).
- Consumers get data **instantly** with **no ETL**: it appears as a read-only database in their account and is always current.
- Consumers pay their own compute plus any listing price.

⭐ Snowflake Native App Framework

- Build an **application** (code + optional data + UI) that **installs and runs inside the consumer's account**, so the consumer's data **never leaves** their account.
- **Provider side:** an **application package** containing a `manifest.yml`, a **setup script** (`setup.sql`, creates the app's objects and **application roles**), **versions and patches** (release directives), optional **Streamlit UI**, Snowpark/UDF code, shared data, and optionally containers (SPCS).
- Published through a **listing** (private or Marketplace), free or paid.

- **Consumer side:** installing creates an **APPLICATION** object. The consumer grants the app privileges it requests (e.g., access to a table through **references**) and controls what it can see.
- Provider **code and IP are protected** (hidden from the consumer). Optional **event sharing** sends logs to the provider for support.
- Examples: Snowflake connectors (ServiceNow, Google Analytics), clean rooms, data-enrichment apps.

✅ Domain 5: must remember

- **Time Travel:** default 1 day; Standard max 1; **Enterprise max 90**; transient/temporary max 1. `AT | BEFORE` with `TIMESTAMP`, `OFFSET` (seconds, negative), `STATEMENT` (query ID). `UNDROP` needs the name free.
- **Fail-safe: 7 days**, permanent tables only, **Snowflake Support only**.
- **Cloning:** zero-copy, independent. Internal named stages need `INCLUDE INTERNAL STAGES`; internal-stage pipes skipped; tasks suspended; **stream records not carried**; single-table clones copy grants only with `COPY GRANTS`.
- **Replication:** databases & shares in all editions; **failover, client redirect, account-object replication = Business Critical**. Secondaries are read-only. Costs = transfer + compute + storage.
- **Sharing:** no copy; provider pays storage, consumer pays compute; read-only; **reader account = provider pays, query only**; direct shares **same region**; **secure views**; `IMPORTED PRIVILEGES`; database roles for fine-grained access; consumers **can't reshare** direct shares.
- **Listings:** private vs public (Marketplace); free / limited trial / paid; cross-region via **Cross-Cloud Auto-Fulfillment**.
- **Native Apps:** application package, `manifest.yml`, setup script, versions/patches, run **in the consumer's account**.
- **Clean rooms:** analyse joint data with approved templates + privacy policies; analysis runner pays.

---

6. Cheat sheets

⭐ Numbers to memorise

Fact	Value
Micro-partition size	**50–500 MB uncompressed**
Warehouse credits/hour	XS 1, S 2, M 4, L 8, XL 16, 2XL 32, 3XL 64, 4XL 128, 5XL 256, 6XL 512
Billing minimum	**60 seconds** per start/resume, then per second
Cloud services free allowance	**10%** of daily warehouse credits
Default AUTO_SUSPEND	**600 s** (10 min)
Default MAX_CONCURRENCY_LEVEL	**8**
Default STATEMENT_TIMEOUT_IN_SECONDS	**172,800 s** (2 days)
Economy scaling rule	Enough load for **6 minutes**
Time Travel	Default **1**; Standard **0–1**; Enterprise **0–90**; transient/temp **0–1**
Fail-safe	**7 days** (permanent only)
Stream staleness extension	`MAX_DATA_EXTENSION_TIME_IN_DAYS` default **14**
COPY load metadata	**64 days**
Snowpipe load history	**14 days**; `ALTER PIPE … REFRESH` looks back **7 days**
`FILES =` list limit	**1,000** files
Recommended file size	**100–250 MB compressed**
Unload default file size	**16 MB**; `SINGLE = TRUE` max **5 GB**
Snowsight upload limit	**250 MB** per file
VARIANT / VARCHAR max	**128 MB** (older material: 16 MB)
Result cache	**24 h**, extended per reuse up to **31 days**
Scoped URL	Expires in **24 h**
Key rotation / rekeying	Every **30 days** / data older than **1 year** (Enterprise)
Key-pair minimum	**RSA 2048-bit**
Resource monitor actions	**5** NOTIFY, **1** SUSPEND, **1** SUSPEND_IMMEDIATE; reset **00:00 UTC**
ACCOUNT_USAGE	**365 days**, **45 min–3 h** latency
INFORMATION_SCHEMA	**7 days–6 months**, real-time
QUERY_ATTRIBUTION_HISTORY latency	up to **8 h**
Dynamic table minimum TARGET_LAG	**60 seconds**
Task graph size	up to **1,000** tasks
QAS default scale factor	**8** (new Gen2/multi-cluster: **2**)
Clustering key columns	**3–4** max recommended
Multi-cluster max clusters	Up to **300** (XS–M) … **10** (4XL+); Snowsight shows 10
Snowpipe price	**0.0037 credits/GB**
Certification validity	**2 years**

⭐ "Which feature?" scenarios

Scenario	Answer
One slow complex query spilling to disk	**Scale up** the warehouse
Hundreds of BI users, queries queuing	**Multi-cluster warehouse** (scale out), auto-scale
Some queuing is OK, minimise credits	**Economy** scaling policy
Occasional huge ad-hoc scans on a right-sized warehouse	**Query Acceleration Service**
Look up one customer by ID in a 10 TB table	**Search Optimization**
Dashboards filter a multi-TB table by date range	**Clustering key** (e.g., on date)
Same aggregation on one big table runs all day	**Materialized view**
Multi-table SQL pipeline that should stay ~10 min fresh	**Dynamic table** with `TARGET_LAG = '10 minutes'`
React to changed rows with custom MERGE logic	**Stream + task**
Files land in S3 continuously	**Snowpipe** (auto-ingest via SQS)
Kafka/IoT rows, seconds of latency	**Snowpipe Streaming** / Kafka connector
Large nightly batch of files	**COPY INTO** on a warehouse
Check files for errors without loading	`VALIDATION_MODE`
Remove files after loading	`PURGE = TRUE`
Reload files already loaded	`FORCE = TRUE` (or TRUNCATE resets metadata)
Access S3 without keys in SQL	**Storage integration**
Run SQL scripts from GitHub	**Git repository** + API integration, `EXECUTE IMMEDIATE FROM`
Hide PII from most roles	**Masking policy** (Enterprise)
Users see only their region's rows	**Row access policy** (Enterprise)
Allow joins on a column but never show it	**Projection policy**
Only allow aggregated results with ≥ N rows	**Aggregation policy**
Find sensitive columns automatically	**Data classification**
Check security posture / CIS benchmark	**Trust Center**
Customer controls the encryption key	**Tri-Secret Secure** (Business Critical)
Allow only office IPs	**Network policy**
Service account login without password	**Key-pair** (or OAuth / PAT), user `TYPE = SERVICE`
Stop a warehouse at a credit limit	**Resource monitor** (SUSPEND)
Track serverless/AI spend against a limit	**Budget**
Who queried this column last month?	**ACCESS_HISTORY** (Enterprise)
Credits per user/team per query	**QUERY_ATTRIBUTION_HISTORY** + `QUERY_TAG`
Notify Slack when a condition is true	**Alert** + webhook **notification integration**
Restore a table dropped yesterday	`UNDROP TABLE` (Time Travel)
Data lost 2 days after Time Travel ended	**Fail-safe** via Snowflake Support
Instant dev copy of production	**Zero-copy clone**
DR across regions with automatic client switch	**Failover group + client redirect** (Business Critical)

Scenario	Answer
Share with a partner in the same region	**Direct share**
Share with a partner in another cloud/region	**Listing + Cross-Cloud Auto-Fulfillment** (or replication)
Partner has no Snowflake account	**Reader account** (provider pays)
Sell data publicly	**Paid Marketplace listing**
Joint analysis without exposing raw data	**Data clean room**
Ship an app that runs in the customer's account	**Native App**
Ask questions of tables in plain English	**Cortex Analyst** (semantic view)
RAG chatbot over documents	**Cortex Search** + `AI_COMPLETE`
Classify support tickets in SQL	`AI_CLASSIFY`
Forecast sales in SQL without ML code	**ML Function** `FORECAST`
Train an ML model needing lots of memory	**Snowpark-optimized warehouse** / Container Runtime
Python data app for business users	**Streamlit in Snowflake**

⭐ Commonly confused pairs

A	B
Scale up = bigger warehouse (complex queries, spilling)	**Scale out** = more clusters (concurrency, queuing)
Standard policy = performance	**Economy** = saves credits
Transient = no Fail-safe, persists	**Temporary** = no Fail-safe, session only
Time Travel = self-service, 0–90 days	**Fail-safe** = Support only, 7 days
Secure view = hides definition, may be slower	**Materialized view** = stores results, Enterprise, single table
Materialized view = always current, no joins, serverless	**Dynamic table** = target lag, joins OK, your warehouse
Snowpipe = files, ~1 min, 14-day history	**Snowpipe Streaming** = rows, seconds
COPY default `ABORT_STATEMENT`	**Snowpipe** default `SKIP_FILE`
`ALL TABLES IN SCHEMA` = existing	`FUTURE TABLES` = not yet created
Account role = can `USE ROLE`	**Database role** = can't be activated directly
Primary role = creates/owns objects	**Secondary roles** = extra privileges for other actions
ACCOUNT_USAGE = 1 year, dropped objects, latency	**INFORMATION_SCHEMA** = real-time, short history
LOAD_HISTORY = COPY only	**COPY_HISTORY** = COPY + Snowpipe
Resource monitor = warehouses, can suspend	**Budget** = all services incl. serverless, notifies
Clustering = range filters, big tables	**Search optimization** = point lookups, few rows
Result cache = cloud services, 24 h	**Warehouse cache** = local SSD, lost on suspend
Scoped URL = 24 h, only generating user	**Pre-signed URL** = anyone, custom expiry; **file URL** = permanent, needs privileges
`TRUNCATE` = fast, clears load metadata	`DELETE` = slower, keeps load metadata
Owner's rights procedure (default)	**Caller's rights** (`EXECUTE AS CALLER`)

A	B
Direct share = same region	**Listing** = any region, Marketplace, paid
Reader account = provider pays	**Full consumer account** = consumer pays compute
Snowflake CLI (`snow`) = current	**SnowSQL** = legacy
Cortex Search = unstructured text retrieval	**Cortex Analyst** = structured data, text-to-SQL

⭐ Who can do what (default roles)

Task	Role
Create accounts in the organization, enable replication	**ORGADMIN / GLOBALORGADMIN**
Create resource monitors, view billing, account parameters	**ACCOUNTADMIN**
Grant/revoke any privilege, create network policies	**SECURITYADMIN**
Create users and roles	**USERADMIN**
Create warehouses and databases	**SYSADMIN**
Create integrations (storage, API, security, notification)	**ACCOUNTADMIN** (or `CREATE INTEGRATION`)
Create a share / import a share	ACCOUNTADMIN (or `CREATE SHARE` / `IMPORT SHARE`)
Run tasks	Owner role with `EXECUTE TASK` (+ `EXECUTE MANAGED TASK` for serverless)
Query ACCOUNT_USAGE	ACCOUNTADMIN by default (grant `IMPORTED PRIVILEGES` on SNOWFLAKE or SNOWFLAKE database roles)
Use Cortex AI functions	`SNOWFLAKE.CORTEX_USER` (PUBLIC by default)

7. Official sample questions — explained

Q1. Procedure succeeds for analysts without SELECT on the table; new rule says it must fail for such callers. Answer: B, re-create it with `EXECUTE AS CALLER`. Procedures run with **owner's rights by default**, which is why analysts could read the table. Caller's rights make the procedure use the caller's privileges. Granting SELECT (A) does the opposite of the requirement. Transferring ownership (C) changes who owns it, not the execution model. A secure view (D) doesn't make the call fail.

Q2. A schema clone contains a stream with thousands of unconsumed records, but the cloned stream returns nothing. Answer: C. A cloned stream starts at the point of cloning; **unconsumed records aren't carried over**. Streams *are* cloned (A is wrong), and the clone's offset is independent of the original (B is wrong).

Q3. Multi-cluster BI warehouse; a few seconds of queuing is fine; minimise active clusters. Answer: B, Economy. Economy only starts a cluster when there's enough work to keep it busy (about 6 minutes), favouring cost over latency. Standard starts clusters as soon as queries queue.

Q4. Nightly `DELETE FROM staging_table` is slow; retention is 0 and Time Travel isn't needed. Answer: A, `TRUNCATE TABLE`. It removes all rows as a fast **metadata** operation. DROP + CREATE (B) loses grants and definitions. Clustering (C) and batching (D) don't fix a full-table delete.

Q5. Stage keeps six months of loaded files; add one option that removes them without changing error handling or reloading files. Answer: D, `PURGE = TRUE`. It deletes files after a successful load. `ON_ERROR` (B) changes error handling, `FORCE` (C) reloads files, and `AUTO_INGEST` (A) is a pipe property.

---

8. Practice questions

Answers are after question 40. Choose one unless stated.

1. Which layer performs query optimization and stores the result cache?
   A) Storage B) Compute C) Cloud services D) Client
2. A Medium warehouse resumes, runs a 20-second query and suspends. How many seconds are billed?
   A) 20 B) 60 C) 300 D) 3,600
3. Which edition is the minimum for a 30-day Time Travel retention?
   A) Standard B) Enterprise C) Business Critical D) VPS
4. Which TWO features require Business Critical? (Choose two.)
   A) Tri-Secret Secure B) Materialized views C) Failover/failback D) Masking policies E) Multi-cluster warehouses
5. A query shows large "Bytes spilled to remote storage". Best first fix?
   A) Add a clustering key B) Increase warehouse size C) Add clusters D) Enable search optimization
6. Fifty analysts' queries queue on a Large warehouse every morning. Best fix?
   A) Resize to 2XL B) Multi-cluster auto-scale C) Enable QAS D) Lower AUTO_SUSPEND
7. What's the size range of a micro-partition?
   A) 16 MB compressed B) 50–500 MB uncompressed C) 100–250 MB compressed D) 1 GB
8. Which table type has no Fail-safe and persists after the session ends?
   A) Permanent B) Temporary C) Transient D) External
9. A parameter is set at account level to 30 and the session sets it to 10. Which value applies in the session?
   A) 30 B) 10 C) The lower of the two always D) An error is raised
10. Which statement about database roles is TRUE?
    A) They can be activated with USE ROLE B) They can be granted to a share C) They are account-level D) They can own warehouses
11. Which role, by default, holds MANAGE GRANTS?
    A) SYSADMIN B) USERADMIN C) SECURITYADMIN D) PUBLIC
12. A user activates secondary roles and runs CREATE TABLE. Which role owns the new table?
    A) The secondary role with CREATE privilege B) The primary role C) PUBLIC D) SYSADMIN
13. Both an account-level and a user-level network policy exist. Which applies to that user?
    A) Account-level B) User-level C) Both combined D) Neither
14. Which authentication method suits an ETL service user that must not use a password?
    A) Password + Duo B) Key-pair C) SAML SSO D) Session policy
15. Which is evaluated first when a table has both?
    A) Masking policy B) Row access policy C) Whichever was created first D) Aggregation policy
16. Which policy lets a column be used in JOIN and WHERE but never returned in SELECT output?
    A) Masking B) Aggregation C) Projection D) Row access
17. A resource monitor should stop running queries the moment a quota is hit. Which action?
    A) NOTIFY B) SUSPEND C) SUSPEND_IMMEDIATE D) ABORT
18. Which can a resource monitor NOT control?
    A) A warehouse B) Several warehouses C) Snowpipe credits D) The whole account's warehouses
19. Which view includes dropped objects and keeps one year of history?
    A) INFORMATION_SCHEMA.TABLES B) ACCOUNT_USAGE.TABLES C) SHOW TABLES D) INFORMATION_SCHEMA.QUERY_HISTORY()
20. Which file format can NOT be used to unload data?
    A) CSV B) JSON C) Parquet D) Avro
21. What is the default ON_ERROR for a bulk COPY INTO a table?
    A) CONTINUE B) SKIP_FILE C) ABORT_STATEMENT D) SKIP_FILE_10%

22. You want to check staged files for errors without loading them. Which option?
    A) VALIDATION_MODE B) FORCE C) PURGE D) LOAD_UNCERTAIN_FILES
23. How long does Snowflake keep COPY load metadata for a table?
    A) 7 days B) 14 days C) 64 days D) 365 days
24. Which TWO are NOT supported in COPY transformations? (Choose two.)
    A) Casting columns B) WHERE filter C) Reordering columns D) JOIN E) Omitting columns
25. Which stage can't be used with COPY transformations?
    A) Named internal B) Named external C) Table stage D) All support them
26. Files arrive in S3 every few minutes and must load automatically with no warehouse. Best option?
    A) Task with COPY B) Snowpipe auto-ingest C) External table D) Dynamic table
27. A stream is queried with SELECT inside a transaction that then rolls back. What happens to its offset?
    A) Advances B) Doesn't advance C) Resets to table creation D) Stream becomes stale
28. Which stream type works on external tables?
    A) Standard B) Append-only C) Insert-only D) Delta
29. A dynamic table uses TARGET_LAG = DOWNSTREAM and nothing reads from it. What happens?
    A) Refreshes every 60 s B) Never refreshes automatically C) Error at creation D) Refreshes daily
30. After CREATE TASK, the task doesn't run on schedule. Most likely reason?
    A) Missing CRON B) Tasks are created suspended C) Needs a stream D) Warehouse too small
31. Which object lets an external stage access S3 without storing keys in SQL?
    A) API integration B) Storage integration C) Network rule D) Secret
32. Which is best for selective equality lookups on a high-cardinality column in a huge table?
    A) Clustering key B) Search optimization C) Materialized view D) Larger warehouse
33. Which TWO are limitations of materialized views? (Choose two.)
    A) No joins B) Requires a warehouse to refresh C) No window functions D) Not available in Enterprise E) Never current
34. Which query can reuse the result cache from an identical earlier query?
    A) One that calls RANDOM() B) One run after the table had an INSERT C) Exactly the same text, unchanged data, same privileges D) Same query with an added alias
35. Which cache is lost when a warehouse suspends?
    A) Result cache B) Metadata cache C) Warehouse (local disk) cache D) None
36. Which FLATTEN argument keeps rows whose array is empty?
    A) RECURSIVE B) OUTER C) MODE D) PATH
37. Which URL type lets someone without a Snowflake login download a staged file for a limited time?
    A) Scoped URL B) File URL C) Pre-signed URL D) Stage URL
38. Which is TRUE of zero-copy cloning a schema?
    A) Internal named stages are always cloned B) Tasks are cloned suspended C) Stream records are preserved D) Grants on the schema itself are copied
39. A consumer in another region needs your data. Which approach works?
    A) Direct share B) Listing with Cross-Cloud Auto-Fulfillment C) Reader account in your region D) Clone
40. Who pays for compute in a reader account?
    A) The reader B) The provider C) Snowflake D) Split equally

Answers

#	Ans	Why
1	C	Cloud services handles optimization, metadata and the result cache
2	B	60-second minimum each time a warehouse resumes
3	B	Enterprise allows up to 90 days

#	Ans	Why
4	A, C	MV, masking and multi-cluster are Enterprise
5	B	Spilling = memory pressure → scale up
6	B	Queuing from concurrency → scale out
7	B	50–500 MB of uncompressed data
8	C	Transient persists, no Fail-safe
9	B	Most specific level (session) wins
10	B	Database roles can be granted to shares; can't be activated directly
11	C	SECURITYADMIN has MANAGE GRANTS
12	B	Only the primary role authorises CREATE and owns the object
13	B	User-level overrides account-level
14	B	Key-pair is the standard for service users
15	B	Row access policies are evaluated before masking
16	C	Projection policy
17	C	SUSPEND_IMMEDIATE cancels running queries
18	C	Monitors cover warehouses, not serverless features; use budgets
19	B	ACCOUNT_USAGE has dropped objects and 365 days
20	D	Unload supports CSV, JSON, Parquet only
21	C	ABORT_STATEMENT (Snowpipe uses SKIP_FILE)
22	A	VALIDATION_MODE validates without loading
23	C	64 days (Snowpipe: 14)
24	B, D	No WHERE or JOIN in COPY transformations
25	C	Table stages don't support COPY transformations
26	B	Snowpipe auto-ingest is event-driven and serverless
27	B	Only a committed DML consumption advances the offset
28	C	Insert-only streams for external tables
29	B	DOWNSTREAM with no consumers never refreshes automatically
30	B	Tasks start suspended; ALTER TASK … RESUME
31	B	Storage integration
32	B	Search optimization for point lookups
33	A, C	MVs are single-table, no window functions; refresh is serverless and results are always current
34	C	All reuse conditions met
35	C	The local SSD cache is dropped on suspend
36	B	OUTER => TRUE
37	C	Pre-signed URLs need no authentication
38	B	Cloned tasks are suspended; internal stages need INCLUDE INTERNAL STAGES; stream records aren't kept; container grants aren't copied
39	B	Direct shares are same-region only

#	Ans	Why
40	B	The provider pays for reader account compute

---

9. Last-48-hours checklist

Can you explain, without notes...

- [ ] The three layers, what each does, and the 10% cloud services rule
- [ ] Which features need Enterprise vs Business Critical
- [ ] Warehouse credits by size, per-second billing, 60-second minimum, Gen2 and Snowpark-optimized
- [ ] Scale up vs out; Standard vs Economy; maximized vs auto-scale
- [ ] Micro-partitions, pruning, clustering depth, when to add a clustering key
- [ ] All table types and view types with Time Travel / Fail-safe
- [ ] Iceberg: external volume, catalog integration, Snowflake vs external catalog
- [ ] Cortex AI functions, Cortex Search vs Cortex Analyst, Snowflake ML pieces, Notebooks, Streamlit, Snowpark
- [ ] Snowsight areas, Snowflake CLI vs SnowSQL, VS Code extension
- [ ] Parameter precedence and the object hierarchy
- [ ] System roles, database roles, secondary roles, future grants, managed access schemas
- [ ] Network policy precedence; MFA, SSO, OAuth, key-pair, user types
- [ ] Masking vs row access vs aggregation vs projection policies; tags; classification
- [ ] Trust Center, encryption hierarchy, rotation, rekeying, Tri-Secret Secure
- [ ] Alerts, notification integrations, lineage, ACCESS_HISTORY, event tables
- [ ] Resource monitors vs budgets; ACCOUNT_USAGE vs INFORMATION_SCHEMA
- [ ] Stages, PUT/GET, file formats, COPY options, load metadata, unloading
- [ ] Snowpipe vs Snowpipe Streaming vs COPY; streams; tasks; dynamic tables
- [ ] Storage, API, Git, notification, external access integrations; drivers and connectors
- [ ] Query Profile symptoms and fixes; query insights; QUERY_HISTORY; QUERY_ATTRIBUTION_HISTORY
- [ ] Clustering vs SOS vs MV vs QAS
- [ ] The three caches and result-cache rules
- [ ] Semi-structured querying and FLATTEN; unstructured URLs; window functions and QUALIFY
- [ ] Time Travel, Fail-safe, UNDROP, cloning rules
- [ ] Replication groups vs failover groups vs client redirect
- [ ] Shares, reader accounts, listings, Marketplace, clean rooms, Native Apps

Exam day

- Sleep, water, ID ready; for online proctoring, a clean desk and a tested webcam.
- First pass: answer what you know, flag the rest. Second pass: flagged questions.
- On "choose two/three", count your selections before moving on.
- Watch for **cost vs performance** wording and **edition** clues.

Sources

- SnowPro Core COF-C03 Exam Study Guide (Snowflake, last updated July 8, 2026)
- learn.snowflake.com/en/certifications/snowpro-core-c03
- docs.snowflake.com pages on: editions; Gen2 and Snowpark-optimized warehouses; multi-cluster warehouses; access control; MFA; network policies; resource monitors; Trust Center; privacy policies; Cortex AI functions, Cortex Search, Cortex Analyst; Notebooks; Snowflake CLI; Iceberg tables; dynamic tables; Snowpipe costs; Snowpipe Streaming; streams; tasks; CREATE STAGE; unstructured data; Git integration; query insights; QUERY_ATTRIBUTION_HISTORY; persisted query results; semi-structured data types; cloning considerations; replication and failover; secure data sharing; listings; data clean rooms.

Snowflake changes quickly. If a detail here conflicts with the current docs, trust the docs. The exam tests features that are generally available.